XHarvest: Rethinking High-Performance and Cost-Efficient SSD Architecture with CXL-Driven Harvesting

Li Peng
Peking University
Beijing, China
idvz@foxmail.com

Wenbo Wu
Huazhong University of Science and Technology
Wuhan, China
709782717@qq.com

Shushu Yi
Peking University
Beijing, China
firnyee@gmail.com

Xianzhang Chen
Chongqing University
Chongqing, China
xzchen@cqu.edu.cn

Chenxi Wang
Institute of Computing Technology, Chinese Academy of Sciences
Beijing, China
wangchenxi@ict.ac.cn

Shengwen Liang
Institute of Computing Technology, Chinese Academy of Sciences
Beijing, China
liangshengwen@ict.ac.cn

Zhe Wang
Institute of Computing Technology, Chinese Academy of Sciences
Beijing, China
wangzhe12@ict.ac.cn

Nong Xiao
Sun Yat-sen University
Guangzhou, China
Xiaon6@mail.sysu.edu.cn

Qiao Li
Xiamen University
Xiamen, China
qiaoli045@gmail.com

Mingzhe Zhang
Institute of Information Engineering, Chinese Academy of Sciences
Beijing, China
zhangmingzhe@iie.ac.cn

Jie Zhang
Peking University
Beijing, China
jiez@pku.edu.cn

Abstract

The occasional nature of I/O bursts in production clusters makes the substantial and expensive SSD internal hardware resources (e.g., computation and memory resources) always underutilized, resulting in cost inefficiency. Open-Channel SSD (OCSSD), as a pioneering solution, removes the SSD internal resources but rather leverages the host-side resources to serve I/O requests. Unfortunately, it faces adoption obstacles due to the heavy resource contention with user applications, hampered host-SSD collaboration, and proprietary firmware leakage risks. Tackling these challenges, we propose XHarvest, a new cost-efficient and high-performance SSD architecture, which harnesses compute express link (CXL) and trusted execution environment (TEE) to facilitate dynamic, efficient, and secure host resource harvesting. It reserves moderate SSD internal resources to isolate SSD internal tasks and applications under regular I/O loads while coping with occasional I/O bursts via dynamic host resource harvesting. To this end, XHarvest executes the firmware within the host-side TEE without disclosing sensitive algorithms while leveraging the cache-coherent and fine-grained CXL to build a unified and efficient metadata cache. XHarvest also capitalizes on the CXL and its security feature to enable secure and efficient host-SSD collaboration. The evaluation results show that XHarvest reduces the hardware cost by 31.50% while achieving the same or even higher performance than conventional SSDs. It relieves resource contention via dynamic harvesting, reducing 2.27× execution time over OCSSD in memory-intensive tasks.

CCS Concepts

• **Hardware** → **External storage**; • **Information systems** → **Storage architectures**.

Keywords

SSD Architecture, Compute Express Link, Trusted Execution Environment

ACM Reference Format:
Li Peng, Wenbo Wu, Shushu Yi, Xianzhang Chen, Chenxi Wang, Shengwen Liang, Zhe Wang, Nong Xiao, Qiao Li, Mingzhe Zhang, and Jie Zhang. 2025. XHarvest: Rethinking High-Performance and Cost-Efficient SSD Architecture with CXL-Driven Harvesting. In *Proceedings of the 52nd Annual International Symposium on Computer Architecture (ISCA '25), June 21–25, 2025, Tokyo, Japan.* ACM, New York, NY, USA, 16 pages. https://doi.org/10.1145/3695053.3731028

ISCA '25, Tokyo, Japan

ACM ISBN 979-8-4007-1261-6/25/06
https://doi.org/10.1145/3695053.3731028

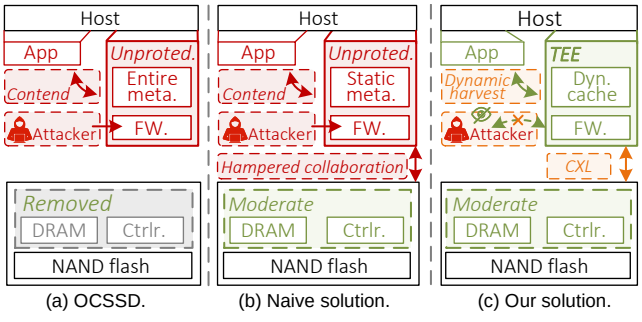


Figure 1: Comparison of various SSD designs.

1 Introduction

Solid-state drives (SSDs) have played a key role of high-performance storage (i.e., burst buffer) in diverse computing domains such as datacenters and high-performance computers [9, 35, 59, 92]. These domains continually demand high I/O throughput for rapid access to ever-increasing datasets [54, 91]. To satisfy this demand, the state-of-the-art SSD devices aggregate numerous storage resources, including hundreds of NAND flash dies operating over up to 16 flash channels simultaneously, delivering 10s GB/s bandwidth [34, 71].

In addition to the abundant storage resources, the high-performance SSD necessitates more computation resources to accelerate the firmware execution such as the flash translation layer (FTL) [11, 93]. For example, PCIe 5.0 SSDs require 4.43× more computing power to achieve 14 GB/s I/O bandwidth compared to PCIe 3.0 SSDs that deliver 3.5 GB/s throughput [22, 109, 110, 112]. Moreover, the SSDs accommodate the entire metadata in their internal DRAM for instant visits, which, unfortunately, requires extensive DRAM capacity (e.g., 10 GB DRAM for a 7.5 TB SSD [17]). However, the considerable integrated resources plunge the future high-performance SSD design into a *cost-utilization dilemma*: While SSD vendors reserve substantial and costly resources (e.g., 30% and 10% cost for computation and DRAM, respectively [25, 65, 70, 87, 108, 117]) in storage devices to meet the I/O bursts from computing domains, the occasional nature of I/O bursts precipitates severe resource underutilization [116]. For example, a long-term study in Alibaba [2, 21] observes that over 96.64% of the operating time, I/O bandwidth utilization on their servers is below 25%, indicating that SSD internal costly computation and DRAM resources are often idle.

This dilemma drives future SSD designs towards both *cost-efficient* and *high-performance*, that is, to reduce underutilized resources and their associated costs without sacrificing the high performance required for occasional I/O bursts. One of the pioneering architectures to strive towards such a goal is *Open-Channel SSD* (*OCSSD*) [8, 55], whose key features are shown in Figure 1a. This prototype completely removes SSD internal computation and DRAM resources but rather leverages the host-side resources to execute the firmware and manage the metadata via the open-source implementation in the Linux kernel (i.e., *LightNVM* and *pblk*) [8]. However, this architecture faces adoption obstacles as the firmware execution and metadata management severely contend with user applications in terms of both CPU and host memory resources [83, 131].

A naive solution to alleviate this resource contention is extending the existing OCSSD design by provisioning moderate computation and DRAM resources within the SSD (cf. Figure 1b). Ideally, this new architecture allows the SSD internal resources to directly serve the regular I/O traffic while exploiting only idle host resources for I/O bursts. This architecture thus reduces the occupancy of the host-side resources, which in turn mitigates resource contention with user applications. However, this naive solution still faces prominent challenges before it becomes practical. *(1) Trade-off between memory contention and cost efficiency:* User applications typically demand substantial memory to store massive datasets for fast access. For example, applications in Alibaba clusters often consume over 95% of host memory capacity at low I/O traffic [2, 21]. However, the naive solution, which inherits the static memory reservation scheme from the OCSSD, always competes with the user applications for the limited memory resources. The penalty of such memory contention drastically depends on the amount of dedicated DRAM resources provisioned in the SSD. Specifically, provisioning more DRAM resources can reduce the occupancy of host memory and further mitigate contention but incur higher costs, whereas allocating fewer resources improves cost efficiency but intensifies contention. How to make a balance between memory contention and cost efficiency remains a serious problem. This trade-off becomes more sophisticated in multi-SSD deployments due to the aggregated memory reservation of all SSDs. *(2) Hampered host-SSD collaboration:* While the naive solution requires tight collaboration between the host and SSD to serve I/O bursts, the existing PCIe interconnect has become the prominent obstacle to achieving this goal. Specifically, the PCIe interconnect separates the host and the SSD into different cache-coherency domains [33, 56]. The host cannot access the SSD internal resources (e.g., retrieve the metadata from the internal DRAM) without the assistance of the complex OS stack. *(3) Compromised host CPU exploitation:* This naive OCSSD-like solution is also vulnerable to security issues. In particular, the unprotected firmware execution on the host faces sensitive firmware leakage risks, forcing SSD vendors to resist integrating their proprietary optimization algorithms and mechanisms into the OS kernel. This resistance limits future performance and feature enhancements for this revised SSD architecture, hindering its broader adoption.

We characterize the resource utilization of typical working scenarios and highlight the emerging techniques that provide new opportunities to address the aforementioned challenges. *(1) Host resource harvesting:* The resource usage in datacenters [2, 21] shows that the I/O burst occurs occasionally on a minority of SSDs while the majority remains idle, indicating that only a few SSDs need host memory to buffer their metadata at any given time. In addition, the statistics we analyzed reveal that the host CPU mostly stays at a low utilization when the SSD reaches a high I/O load. These resource utilization characteristics could inspire a dynamic resource allocation scheme (i.e., *harvesting*) to exploit host resources for coping with I/O bursts while alleviating host resource contention. *(2) Compute express link (CXL) interconnection:* The emerging CXL integrates the host CPU and peripherals (e.g., SSDs) within a unified cache-coherency domain and enables both the host CPU and SSD controller to directly access the unified memory space via simple load/store instructions [15, 57]. This tight integration can streamline the host-SSD collaboration. *(3) Trusted execution environment (TEE):* TEE techniques, such as Intel's software guard extensions (SGX) [5, 44], offer a promising solution to the firmware leakage

risk, which enables secure program execution in an isolated environment (i.e., enclave) on the host. By executing sensitive algorithms in the enclave, SGX protects them from unauthorized access by any software or even the OS, thus avoiding the potential leakage risk.

Inspired by the above insights, we propose *XHarvest*[1], a new cost-efficient and high-performance SSD architecture (cf. Figure 1c), which harnesses CXL and SGX to facilitate dynamic, efficient, and secure host resource harvesting. Specifically, following the design principle of the naive solution, XHarvest retains moderate computation and DRAM resources within the SSD, which ensures isolation between SSD internal tasks and applications under regular I/O loads. XHarvest then removes the remaining underutilized internal resources in the conventional SSD, thereby ensuring cost efficiency. As the I/O load increases (i.e., I/O burst), XHarvest dynamically harvests host resources to sustain high I/O performance. It executes the firmware within the enclave, which exploits the powerful host CPU to cope with I/O bursts while protecting sensitive algorithms. Moreover, XHarvest capitalizes on the CXL interconnection to enable direct access between the host CPU and SSD internal DRAM, thereby bypassing the intricate OS stack and simplifying their collaboration. To further secure the enclave-SSD collaboration, it leverages a newly introduced CXL feature, called *TEE security protocol*, to permit only the authenticated enclave to access the internal DRAM, thus barring potential attackers. By utilizing this secure memory semantic, XHarvest builds a secure and efficient message-based communication mechanism for enclave-SSD co-execution. To harvest memory, XHarvest employs the cache-coherent and fine-grained interface provided by CXL to construct a unified and efficient metadata cache that spans the host memory and internal DRAM. XHarvest then integrates a host-SSD coordination framework to exploit resources on both sides. Our evaluation results show that XHarvest achieves 31.50% cost savings and 5.02% higher throughput over conventional SSDs. Its dynamic harvesting relieves memory contention, reducing 2.27× execution time and decreasing 10.98% tail latency over OCSSD in memory-intensive tasks.

Our main **contributions** can be summarized as follows:

• *Rethinking cost-efficient and high-performance SSD:* Emerging OCSSD-like architectures incur resource contention due to their static host-side resource occupancy. Our in-depth analysis of host resource utilization characteristics inspires a dynamic harvesting method to ensure stable SSD performance without excessive resource contention. Specifically, we observe occasional I/O bursts in a limited number of SSDs at any given time. It indicates a potential approach to reduce simultaneous memory use if dynamically allocating host memory to the SSDs on demand. Moreover, the trends in CPU utilization suggest that SSDs could harness unused host CPU during high I/O loads without affecting application performance. Based on the analysis, we propose a new cost-efficient and high-performance SSD architecture called XHarvest. It isolates SSD internal tasks and applications under low I/O loads via moderate internal resource reservation while dynamically harvesting host resources to cope with I/O bursts.

• *Efficient and secure resource harvesting:* Resource harvesting is impeded by the hampered host-SSD collaboration and firmware leakage risks. We utilize the CXL to enhance this collaboration by unifying host memory and SSD internal DRAM while providing fine-grained and cache-coherent memory semantics. Moreover, the TEE technique presents a robust defense against potential attacks on host-based firmware execution. However, it is non-trivial to integrate these two techniques simultaneously. The CXL-enabled unified memory exposes SSD internal DRAM to potential unauthorized access by host-side attackers, resulting in sensitive information leakage. To mitigate this risk, we exploit the TEE security protocol feature to build an authentication mechanism that restricts SSD internal DRAM access to only the trusted enclave. We further design a secure and efficient communication mechanism that empowers the enclave and the SSD to handle I/O bursts cooperatively.

• *Harmonized host-SSD coordination framework:* To facilitate dynamic harvesting under varying I/O loads, we design a host-SSD coordination framework. This framework incorporates the load detection mechanism that swiftly harvests host resources upon detecting I/O bursts and promptly releases them as I/O loads diminish. This mechanism can activate harvesting within 5 ms without affecting the tail latency for prolonged workloads. Note that the SSD controller and host CPU can exploit the unified memory and flash dies to serve requests simultaneously. Given the sharp difference in their computing power, our framework further apportions these resources to each side in alignment with their computing power. It also distributes I/O loads to match their processing capabilities, thereby fully exploiting these resources.

[1]XHarvest is accessible at https://github.com/ChaseLab-PKU/XHarvest.

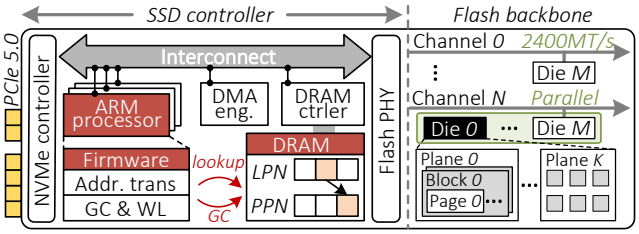


Figure 2: SSD architecture.

2 Background

Figure 2 depicts a typical SSD architecture, which encompasses the flash backbone and the SSD controller [30, 129]. The controller comprises multiple embedded ARM processors, a DRAM controller, an NVMe controller, and specialized processing elements (e.g., DMA engine) [71].

Flash backbone. The flash backbone harnesses the parallelism of NAND flash dies to deliver superior I/O throughput by organizing them into a multi-channel structure [69, 128]. Specifically, each flash channel, in the form of bus-based interconnect, works independently to connect between the SSD controller and a bunch of flash dies for parallel data transfer. A single flash die contains hundreds of flash blocks, each housing several hundred pages.

Firmware. The embedded ARM processors are primarily responsible for executing the flash firmware, including processing the I/O requests from the NVMe controller, forwarding them to the flash backbone, and coordinating the DMA engine to transfer data over the high-speed interconnect interface (i.e., PCIe) between the host and the backbone [99]. Before forwarding requests, the firmware undertakes address translation since the flash intrinsic (i.e., erase-before-write) compels out-of-place updates [72, 124]. To hide this

limit from the host, the firmware builds an indirection layer called *flash translation layer (FTL)*. The FTL maintains a mapping table in SSD internal DRAM to map host logical addresses (LPNs) with flash physical addresses (PPNs) at the indirect unit granularity (typically 4KB~16KB) [11, 93]. During overwriting, it remaps the LPN to a new PPN from a pre-erased block and invalidates the obsolete flash page. When FTL exhausts flash blocks, it invokes wear-leveling (WL) and garbage collection (GC) to reclaim invalid pages.

Resource integration. Technology advances in the Open NAND Flash Interface (ONFi) and PCIe markedly expand external and internal bandwidth. For instance, the PCIe 5.0 SSD can deliver up to 14 GB/s bandwidth, while the ONFi 5.0 evolves the flash backbone from 4 channels at 800 MT/s rate to 16 channels at 2400 MT/s rate [80]. This substantial bandwidth increase requires the firmware to rapidly process firmware tasks (i.e., address translation, WL, and GC), which imposes a considerable computing burden. To meet such demands, SSD vendors conduct significant software and hardware optimization. In fact, PCIe 5.0 SSDs require 4.43× more computing power compared to PCIe 3.0 SSDs [22, 109, 110, 112]. On the other hand, the vendors customize their proprietary firmware designs, including request scheduling schemes, wear-leveling strategies, and GC algorithms. Note that these optimized firmware designs are strictly protected business secrets and cannot be disclosed to others. Moreover, the FTL mapping table imposes substantial memory demands (e.g., 10 GB DRAM for a 7.5 TB SSD [17]).

3 Motivation

3.1 Preliminary studies

Cost-utilization dilemma. While the increasing bandwidth of SSDs demands substantial computation and memory provisioning, these resources are underutilized due to continuously low I/O activity. Figure 3a shows the distribution of I/O bandwidth utilization for 4,000 Alibaba servers during an 8-day resource usage monitoring period [2, 21, 32, 116]. These servers utilize less than 25% of the I/O bandwidth during more than 96.64% of runtime. Furthermore, these servers exploit over 80% of the available I/O bandwidth for only 0.28% of the running time. Similar underutilization occurs within other production clusters like Google and Facebook [51, 90]. This presents a cost-utilization dilemma: peak I/O demands require expensive resource aggregation, but their occasional nature leads to severe resource underutilization. Note that these substantial and expensive computing and memory resources contribute to 30% and 10% of costs for a 1 TB SSD, respectively [25, 65, 70, 108, 117], driving the exploration of the SSD architecture to strive for cost efficiency without compromising high performance.

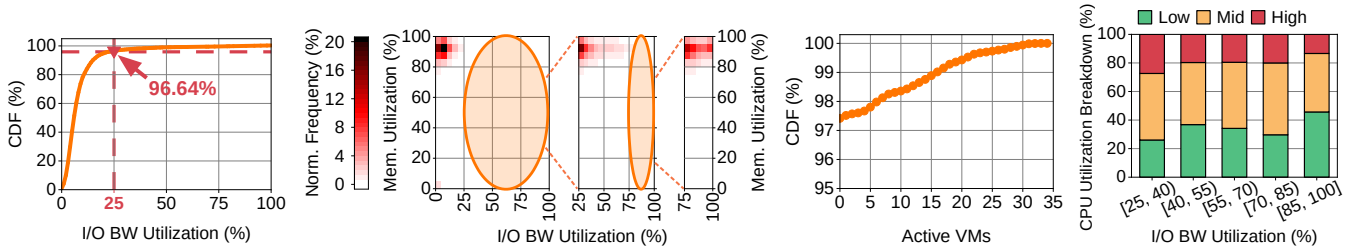
Open-Channel SSD. To this end, recent research proposed a pioneering SSD architecture called *Open-Channel SSD (OCSSD)* [8, 55]. Unlike conventional SSDs, OCSSD (cf. Figure 4) retains only the basic NAND flash control functions (e.g., flash read and write) rather than the complete controller, thereby removing the expensive internal computation and memory resources. OCSSD delegates the flash management to the Linux *LightNVM* subsystem. Atop LightNVM, Linux implements a host-based open-sourced firmware functionality also known as *pblk* [8]. The pblk maintains the entire FTL mapping table in host memory and performs the firmware tasks, which organize the raw flash into a conventional SSD. However, OCSSD simply replaces SSD internal resources with host resources, which instead raises the host-side resource contention issues. In particular, OCSSD statically reserves excessive host memory to accommodate the FTL mapping table. This memory usage becomes more overburdened in the common platform in computing domains where a server deploys multiple large storage devices (e.g., more than 500 TB storage capacity [1, 6]). This significant host memory reservation directly competes with the user applications, as applications are also memory-intensive [29, 96]. As shown in Figure 3b, the memory capacity consumed by applications on the Alibaba servers remains consistently around 90%. Other production clusters show similarly high memory capacity usage [51, 82, 90]. If multiple large-capacity OCSSDs are deployed, the intense memory contention they introduce seriously impacts the application performance [57]. In addition, OCSSD also leads to the CPU contention issue, as applications utilize the host CPU for their computation tasks. We observe that applications in Alibaba servers maintain high CPU utilization during 27.33% of runtime under moderate I/O traffic (cf. Figure 3d), as we soon analyze.

Naive solution. A straightforward idea to alleviate the resource contention is to retain moderate internal DRAM and a weak ARM processor within the SSD. This architecture can exploit both the host CPU and the internal processor to run firmware and buffer the FTL mapping table in both reserved host memory and the internal DRAM. These moderate resource provisions can alleviate the dependency on the host resources while preserving cost efficiency. However, this straightforward solution still faces several challenges, as we analyze shortly.

3.2 Challenges

• *Trade-off between memory contention and cost efficiency:* While the naive solution integrates internal DRAM to alleviate memory contention, how to balance the costs associated with integrated DRAM and the contention remains to be considered. Note that memory-intensive applications require substantial memory, especially consuming more than 95% of the host memory capacity at low I/O traffic (cf. Figure 3b). However, the static memory reservation scheme in the naive OCSSD-like solution precipitates memory contention with user applications since it always occupies memory regardless of I/O loads. Thus, the severity of such contention depends heavily on the amount of DRAM provisioned in the SSD: more DRAM reduces host memory usage and alleviates contention but increases cost, while less DRAM improves cost efficiency but worsens contention. This contention issue is exacerbated due to the aggregated reservation of multiple SSDs.

• *Hampered host-SSD collaboration:* The naive solution suffers from the hampered host-SSD collaboration to leverage both the host-side and SSD internal resources. In the current PCIe interconnection, the SSD internal DRAM resides in a distinct cache-coherency domain separate from the host CPU, which disallows the host to access this DRAM directly. The host must retrieve the metadata in internal DRAM via the complex OS stack, resulting in a significant latency penalty (e.g., 10 us for OS stack [120] vs. 200 ns for a single memory shot [57]). In addition, this non-cache coherency forces the host to copy the metadata to its main memory via DMA,



(a) CDF of I/O BW utilization. (b) Distribution of memory capacity utilization. (c) CDF of active VMs. (d) Distribution of CPU utilization.

Figure 3: Resource utilization characteristics in Alibaba clusters.

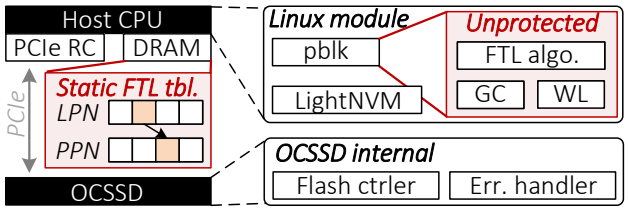


Figure 4: Design details of OCSSD.

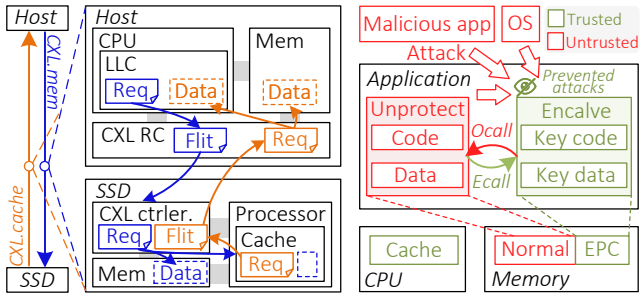
which causes redundant copy and DMA transfer overhead. Furthermore, coarse-grained DMA (e.g., 4KB) is not aligned with the granularity of mapping entries (i.e., 8 bytes), leading to the transfer of unnecessary entries. As shown in Figure 6, this hampered host-SSD collaboration results in 95.81% degradation in FTL processing throughput, especially when stacked with security mechanisms, as we discuss shortly.

• *Compromised host CPU exploitation:* This naive OCSSD-like solution also faces security issues. Specifically, although host-based FTL (e.g., pblk) can exploit the host CPU, this open-source implementation compromises the firmware confidentiality. Without confidentiality safeguards, SSD vendors resist integrating their proprietary optimization algorithms and mechanisms into the OS kernel, which restricts feature and performance enhancements and hinders broader adoption. As a result, the Linux kernel 5.15 deprecates LightNVM and pblk [63].

3.3 Key Insights

We examine resource utilization characteristics of typical datacenter workloads and highlight emerging technologies, deriving key insights to address the aforementioned challenges.

Host resource harvesting. Lessons gained from OCSSD and the naive solution indicate that the static host memory allocation is suboptimal, as it provokes severe resource contention. The *burst interleaving* observed in current production clusters, where not all SSDs experience peak loads simultaneously but in a sporadic pattern, implies a dynamic memory exploitation method to address this issue. As Figure 3c illustrates, the number of active VMs (i.e., I/O bandwidth utilization over 50%) exceeds five concurrently only for 2.20% of the runtime, indicating infrequent simultaneous high loads. The common storage allocation for VMs (i.e., multiple VMs sharing an SSD) further demonstrates the limited simultaneous active SSDs [86]. The burst interleaving indicates slight memory demands concurrently, even for multiple SSDs. Therefore, we can dynamically allocate host memory to SSDs on demand (i.e., memory harvesting) while ensuring low simultaneous memory usage.



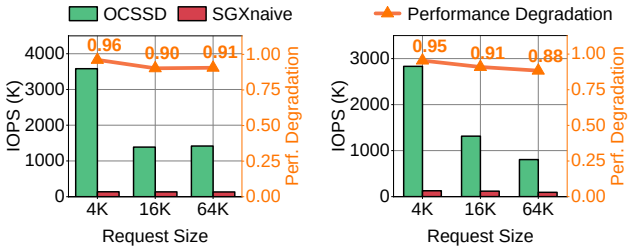
(a) CXL interconnect. (b) Intel SGX.

Figure 5: Details of CXL and SGX.

Figure 3d presents the distribution breakdown of CPU utilization at varying I/O loads, categorizing the utilization into `Low` (0~40%), `Mid` (40~60%), and `High` (60~100%). When the I/O load reaches 25~40%, `Mid` and `High` constitute 46.61% and 27.33%, respectively. As I/O load intensifies to 85~100%, the `Low` becomes predominant at 45.69% due to possible CPU waiting [54, 122]. This presents an opportunity for SSDs to harvest the idle host CPU during high I/O loads. By harvesting host resources during peak loads, the SSD can avoid substantial resource aggregation and only needs to retain moderate resources to serve regular I/O loads.

Compute express link (CXL). The CXL [57], an emerging interconnect standard, reshapes the interaction between host and peripheral devices (e.g., SSDs). Specifically, CXL introduces three protocols, including the I/O semantic protocol (*CXL.io*) and the cache-coherent memory semantic protocols for accessing host memory (*CXL.cache*) and device memory (*CXL.mem*). As shown in Figure 5a, CXL.mem allows the host CPU to map device memory as cacheable memory into the host address space, known as host-managed device memory (HDM) [15, 33, 57]. This enables the host to access the device memory via load and store instructions with fine granularity (i.e., cacheline). Similarly, CXL.cache permits devices to cache the host memory with cache coherency. These protocols not only build a unified memory space with cache coherency between the host and devices but also enable rapid direct memory access without OS intervention. By building upon the high-performance PCIe interfaces, CXL ensures low-latency and high-bandwidth memory access [57, 101, 107]. Moreover, this fine-grained cache-coherent interface facilitates direct data transfer from the device memory into the CPU cache, thereby eliminating unnecessary copy (cf. §3.2).

Trusted execution environment (TEE). Deploying sensitive codes and data (e.g., the SSD firmware and associated metadata) on untrusted platforms raises significant security concerns, such as susceptibility to attacks from compromised OS or malicious entities.



(a) Random read performance. (b) Random write performance.

Figure 6: Throughput comparison of firmware.

To counteract these threats, the industry develops an isolated runtime environment in which applications can be securely executed, known as TEE. The CPU vendors introduce different TEE implementations for their respective computing platforms, such as Intel Software Guard Extensions (SGX), AMD SEV, and ARM TrustZone [44, 58, 103]. Since they offer similar security functionalities, we focus on the Intel SGX for simplicity. As shown in Figure 5b, the application is divided into untrusted and sensitive parts. To protect the sensitive part (i.e., trusted part), SGX provides a protected secure container called an *enclave*, which safeguards sensitive application data and code by encrypting them in a special host memory area called Enclave Page Cache (EPC) during runtime [4, 48, 103]. The processor decrypts these encrypted data when loading them into the CPU cache and encrypts them when writing them to EPC, which prevents other software from accessing the sensitive EPC contents. Additionally, SGX enforces access control to ensure that only the enclave can access its EPC memory. This encryption and access control preserve the confidentiality of the data and code within the enclave. Furthermore, the proof-based attestation offered by SGX [12, 13, 38, 89, 118] ascertains that enclaves are running on a real SGX-enabled processor.

Compared to other security schemes, TEE offers a practical balance between security and performance, making it well-suited for securing host-based firmware execution. Specifically, while hypervisors [62] provide security isolation, they cannot protect SSD firmware from privileged attacks or malicious administrators. Standalone encryption [31] secures data but fails to protect runtime code confidentiality. Oblivious RAM (ORAM) [78] mitigates access pattern leakage and defends against side-channel attacks but incurs prohibitive performance penalties (e.g., up to two orders of magnitude) and still does not protect code secrecy. In contrast, TEE provides a practical security solution to satisfy the security demands of host-based firmware execution with moderate overhead (cf. §6.3 for detailed analysis).

However, our preliminary study reveals that directly porting the FTL algorithm into the enclave to harvest host resources suffers from serious performance issues. As illustrated in Figure 6 (cf. §6 for detailed configuration), this naive SGX solution (`SGXnaive`) results in 96.13% and 95.47% degradation in FTL processing throughput for 4K read and 4K write, respectively, compared to OCSSD. While the gap narrows with larger I/O requests, it remains at 90.59% and 88.39% for 64K read and write. This heavy degradation stems from the interaction between the enclave and the SSD. In particular, since enclaves run at the user level, they cannot directly access peripheral devices (e.g., SSDs) managed by the OS. To interact

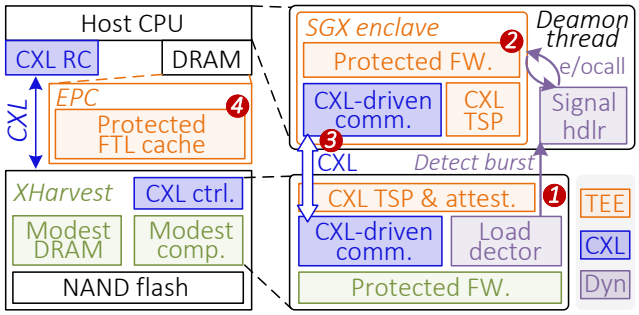


Figure 7: Overview of XHarvest.

with these devices, the enclave must transition to untrusted code through the special function call named *ocall*, and subsequently utilize the traditional OS I/O stack to interact with the SSD. After the interaction, the program returns to the enclave via *ecall*. The ecall and ocall incur more than 20K CPU cycles overhead [121], as they save the register state and other information and perform necessary security checks. Fortunately, the emerging CXL interconnect allows direct access between the hosts and the SSD to streamline their collaboration, and the new TEE security protocol feature in CXL 3.1 ensures a secure data exchange between the SSD and the enclave [16, 28, 33, 66, 106].

4 XHarvest Overview

Inspired by the aforementioned analysis and key insights, we propose *XHarvest*, a new high-performance and cost-efficient SSD architecture (cf. Figure 7). This new architecture shifts from substantial internal resource provisioning to moderate resource provisioning with dynamic host resource harvesting. In particular, XHarvest retains only moderate internal resources within the SSD to ensure base performance during common low I/O loads and removes remaining underutilized resources. By default, XHarvest reserves 25% of the computing power of the conventional SSD and memory that can accommodate 10% of the FTL mapping table. This SSD architecture achieves cost efficiency by removing the costly but underutilized resources. It also mitigates resource contention under low I/O loads by utilizing internal resources to serve I/O requests and thus ensuring isolation from applications. Moreover, to cope with occasional high I/O bursts, we design a secure and efficient harvesting mechanism based on TEE and CXL to exploit host CPU and memory resources.

Workflow of harvesting. XHarvest stores the encrypted firmware binary within the SSD. To enable dynamic resource harvesting during I/O bursts, we incorporate a load detector within the SSD that continuously monitors the I/O load. Upon detecting a peak load, the detector signals a daemon thread on the host, which prompts the thread to launch an enclave and then loads the encrypted firmware (❶, §5.4). After initiating the enclave, the enclave leverages the host computing power to serve most I/O requests (❷, §5.1). This host-based I/O request processing needs efficient collaboration between the enclave and the SSD. To enable this, we design a CXL-driven efficient communication mechanism (❸, §5.2), which allows the enclave to receive requests from the SSD, process them, and transfer the translated addresses back to the flash backbone. Additionally, XHarvest also harvests a portion of host memory resources to build

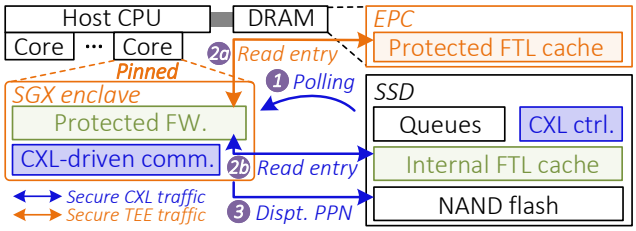


Figure 8: I/O path of CPU harvesting.

a large FTL cache (❹, §5.3). To fully capitalize on the resources of the SSD and host, we design a host-SSD coordination framework (§5.4), which orchestrates memory and flash resource allocation between them and distributes workloads strategically on both sides.

5 Design and Implementation

5.1 Secure CPU Harvesting

SGX enclave launch. XHarvest secures the confidentiality of the firmware by allowing only the authorized enclave to access and execute the encrypted binary code. When the SSD detects a high I/O load, the host daemon thread initiates an enclave to run the initialization function, which is responsible for subsequent firmware loading and decrypting. The SSD then requests the attestation service (e.g., Intel Attestation Service or other trusted service [12, 13, 89, 118]) via the daemon thread, which verifies the integrity of the enclave and certifies that it runs on a genuine SGX processor with the expected initial state. The attestation process incorporates communication encryption to safeguard against potential attacks from a compromised daemon thread [12, 89, 118]. Following successful attestation, the enclave acquires the encrypted firmware binary and the decryption key from the SSD, enabling it to decrypt and execute the firmware. Notably, AI, ML, and big data applications already utilize similar binary encryption and attestation to ensure the confidentiality of their programs and data [5, 81, 89].

I/O path. The enclave is pinned to a single host CPU core to continuously process I/O requests. Figure 8 shows the detailed I/O path. It fetches I/O requests by polling the request queues within the SSD (❶). Note that the NVMe controller in the SSD (cf. Figure 2) parses the NVMe commands and places them in the request queues. Upon fetching a request, the enclave then consults the FTL cache to locate the FTL mapping entry based on the target address of the request (cf. §5.3 for the detailed workflow). If the required mapping entry is already buffered, the enclave can directly access it. Depending on the location of the entry (EPC memory or DRAM inside the SSD), the enclave adopts different access methods. For entries in EPC memory, it moves and decrypts the mapping entry to the CPU last-level cache (LLC) (❷a). Conversely, for entries in the internal DRAM, the enclave leverages our secure CXL traffic (described in §5.2) to securely transfer the entry to the LLC (❷b). If the target mapping entry is not in the FTL cache, the enclave first reads it from the flash. With the retrieved mapping entry, the enclave translates the LPN to the corresponding PPN for the read operation. The write operations require the enclave to allocate a new PPN and update the mapping entry accordingly. Following these steps, the enclave dispatches the PPN to the flash backbone for reading or writing flash (❸). After the flash backbone performs the flash operation, it posts the completed request into the response queue within the SSD, and the request is subsequently delivered to the host. Note that the enclave engages in multiple communications with the SSD, including request retrieval and FTL mapping entry retrieval. Our CXL-driven communication facilitates these interactions efficiently, which avoids the traditional costly ocall and ecall operations.

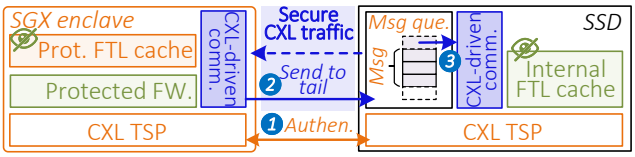


Figure 9: CXL-driven efficient communication.

5.2 CXL-Driven Efficient Communication

The emerging CXL technology reshapes the interaction between the host and peripheral devices (e.g., SSD) by providing low-latency, cache-coherent memory semantics between them [15, 33, 101]. By sharing the SSD internal memory with the host, CXL technology facilitates the exchange of critical metadata (e.g., LPNs, PPNs, FTL mapping entries) through the shared memory space. Nevertheless, a challenge arises from the unrestricted access to the shared memory. The critical metadata within this shared memory is susceptible to attacks since attackers (e.g., compromised OS) on the host can access the shared memory. A straightforward approach to safeguard the metadata is to encrypt the shared memory like SGX EPC. However, this comes with significant en/decryption overhead, even when the SSD firmware accesses the metadata residing in the internal memory. Upon a deeper analysis, we recognize that both the enclave and the SSD are inherently secure and trusted entities. Consequently, storing critical metadata in their memory (i.e., the EPC and SSD internal memory) remains protected without the need for additional encryption. The primary concern then shifts to preserving the confidentiality of the traffic between these trusted entities. The CXL 3.1 specification addresses this concern with the introduction of the TEE Security Protocol (TSP) feature, which encrypts the traffic to prevent physical attacks [16, 33, 66, 106].

Based on this analysis, we build an efficient communication mechanism driven by CXL (cf. Figure 9). After an enclave is created and attested, it conducts mutual authentication with the SSD based on the TSP component measurement and authentication (CMA) [16, 66, 104]. This process involves certificate exchanges, authenticity checks, and identity verifications (❶). Once authenticated, the enclave and SSD securely exchange keys for the AES-GCM symmetric encryption algorithm. This algorithm serves a dual purpose: it encrypts CXL traffic (i.e., CXL flits) to protect its confidentiality during transmission, and this algorithm generates a message authentication code (MAC) to check the integrity of each new CXL flit received (i.e., the flit is not tampered with). This symmetric encryption requires only almost one CPU cycle per byte, thus introducing a mere 5% latency overhead compared to the latency of accessing CXL memory [26, 43, 98]. Following establishing the *secure CXL traffic*, we design a message-passing communication mechanism. Taking communication from the host to the SSD as an example, this mechanism utilizes a portion of SSD internal memory as a ring buffer to accommodate data to be transferred (i.e., messages). We configure the message size as 64 bytes. The host and the SSD maintain the tail and head pointer for the position that produces

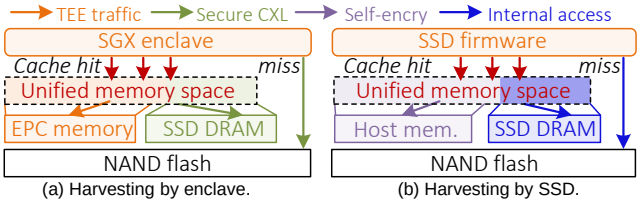


Figure 10: Details of memory harvesting.

and consumes the message, respectively. When the enclave sends a message to the SSD, it constructs the message content at the tail position and then signals message readiness through a specific bit (i.e., ready bit) within the message (❷). Cache coherence provided by CXL ensures that the SSD can observe this message, and the SSD polls the ready bit at the head position to retrieve the message (❸). The reverse communication path (i.e., from the SSD to the enclave) follows a similar procedure. Compared to OS-based interaction, our CXL-based communication mechanism introduces significantly less overhead and offers higher throughput.

5.3 Memory Harvesting

In addition to driving the efficient communication mechanism, CXL allows the host and SSD to access the unified memory with cache-coherent memory semantics, facilitating memory harvesting.

Memory harvesting by enclave. The enclave can access host memory (e.g., EPC) directly as well as internal SSD memory via the secure CXL traffic (cf. §5.2), which enables it to build a combined FTL cache. Specifically, the SSD stores the complete FTL mapping table on special flash pages, known as *translation pages*, each containing multiple mapping entries for adjacent LPNs. Upon initialization, the enclave attempts to allocate a moderate amount of EPC (i.e., harvesting host memory), while also managing a portion of internal memory in the SSD (as detailed in §5.4). The enclave then leverages these two memory spaces to buffer translation pages (cf. Figure 10a). To mitigate the penalty imposed by retrieving translation pages from flash, the enclave employs the least recently used (LRU) replacement policy to manage the FTL cache, exploiting the temporal locality to improve the cache hit ratio [49]. For quick retrieval of translation pages, the enclave maintains a hash table index to track these buffered pages. During the lookup of the mapping entry, the enclave first searches the hash table for the target translation page. A successful search allows the enclave to immediately access the target mapping entry in the page through the load/store instruction, regardless of the location of the entry (in the EPC or SSD internal memory). If the target page is absent in the FTL cache, the enclave evicts a victim page picked by the LRU policy before retrieving the target page into the cache.

Memory harvesting by SSD. The CXL also enables the firmware to construct an FTL cache composed of both the internal memory and the host memory (cf. Figure 10b), which is similar to the FTL cache built by the enclave. However, unlike the enclave which directly utilizes the encrypted memory (i.e., EPC) to ensure the confidentiality of mapping entries, the firmware can only access regular host memory, which lacks such security features. To address this, it individually ensures confidentiality by encrypting buffered mapping entries in the host memory. In addition, it employs integrity verification (e.g., CRC or ECC) to detect metadata corruption [100]. Note that CXL permits memory harvesting by both SSD firmware and the enclave. Due to the superior computing power of the enclave, it is the primary choice for better exploiting host memory, relegating the firmware to a secondary choice when the enclave cannot be initiated due to limited host CPU resources.

5.4 Harmonized Host-SSD Coordination

With the CPU and host memory harvesting, the enclave on the host and the firmware within the SSD can process I/O requests concurrently, which leads to potential resource contention over memory and flash channels. To address this issue, we design a host-SSD coordination framework to coordinate resource allocation between them. Furthermore, the framework includes a dynamic enclave launch mechanism to facilitate on-demand harvesting.

Resource allocation and load distribution. Considering the computing power disparity between the enclave (i.e., a CPU core) and SSD firmware, it is important to allocate appropriate flash and memory resources for them to fully harness their computing power. The framework, therefore, assigns flash channels and memory (including SSD internal memory and host memory) in proportion to their respective computing power (e.g., 1 vs. 6 for the firmware and the enclave in our evaluation). Following this assignment, the enclave and SSD firmware leverage their designated resources independently to serve I/O requests, effectively mitigating the performance issue due to the contention. Note that flash channel partitioning avoids conflicts arising from write requests, while read requests do not follow this partitioning and are instead directed to the flash channel associated with their respective PPNs. Fully utilizing these resources also requires the matching I/O load. To address this, the framework integrates a load partitioning mechanism. This mechanism divides the logical address space into 2MB units, aligning with the continuous address space covered by a single translation page. The mechanism employs a round-robin approach to assign these divided units to the enclave and the firmware based on their resource assignment proportion. For instance, with a resource assignment proportion of 1:6, the mechanism allocates the first unit of every seven to the SSD and the subsequent six units to the enclave. Consequently, the firmware and the enclave handle I/O requests from their designated address spaces. This fine-grained division can distribute the I/O load approximately in proportion to the resources allocated, thereby fully utilizing these resources.

Dynamic enclave launch. We integrate a load detector in the SSD to monitor the I/O load within a time window (e.g., 5 ms). If the load exceeds a predefined threshold (e.g., 60%), the detector activates a flag variable pre-allocated in host memory. A daemon thread on the host polls this flag at a low frequency (e.g., every 1ms), incurring negligible CPU consumption (i.e., 0.1%). Upon flag activation, the daemon thread initiates the enclave. However, due to the time-consuming enclave instantiation, we decouple this process from the critical path to prevent SSD overloads and latency spikes from untimely instantiation. Specifically, at host startup, the daemon thread creates the enclave and loads the firmware, but it does not allocate EPC for the FTL cache to avoid memory contention with applications. It then exits the enclave and resumes polling. When a high load is detected, the thread re-enters the enclave via ecall and allocates the EPC for FTL caching. Once the I/O load decreases, the

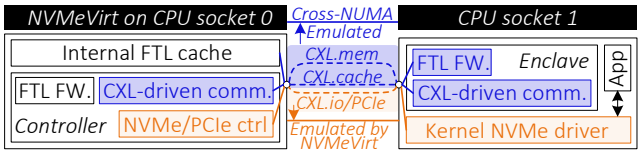


Figure 11: Emulation methodology for XHarvest.

Host System		NVMeVirt		Software	
CPU	2× Intel Xeon 8562Y+	**FW.**	2000K Random IOPS	**Linux kernel**	5.15
	32 cores / 2.8 GHz	**Flash**	Rd./Wr.: 25/50us		
Mem	480GB / DDR5		8 channel / 12 die / 3640	**SGX**	2.25
	local/numa: 102/175ns		block / 512 page / 4 KB	**fio**	3.16
Energy parameter	Flash operation voltage=3.3V, I_{READ}, I_{PROG}, I_{ERASE}=25mA $I_{BUSIDLE}$=5mA, I_{STDBY}=10uA, $CXL/PCIe_{PHY}$=7.5pJ/bit SSD memory read/write=22pJ/bit				

Table 1: System configurations.

enclave gradually flushes back the FTL cache, releases its memory, and exits back to the daemon thread via ocall.

5.5 Implementation

XHarvest strictly follows NVMe and CXL specifications [15, 77] to build its architecture without hardware violations. As CXL.io functions equivalently to PCIe, CXL hardware (SSD CXL controller and CPU home agent) can seamlessly handle CXL.io (PCIe) and CXL.mem/cache requests over the same PHY lanes, forwarding them to appropriate software. The NVMe driver and SSD firmware process CXL.io-based NVMe commands and operate NVMe protocol, including NVMe queue management and DMA transfers, while memory controllers in both the host and the SSD handle CXL.mem/cache requests. These units operate with unified CXL without explicit protocol switching between PCIe and CXL.mem/cache. By leveraging the unified CXL, we use the widely-adopted emulation methodology [57, 61, 130] to capture our design complexities without emulating new hardware out of CXL and NVMe specifications (cf. Figure 11). Specifically, we utilize NUMA to emulate the CXL.mem/cache, mirroring CXL performance while utilizing NVMeVirt [47], a well-recognized tool, to support accurate PCIe and NVMe emulation. The emulated SSD and the enclave are positioned in two sockets, with CXL.mem/cache facilitated by cross-NUMA access, while the CXL.io (PCIe) is emulated by the accurate NVMeVirt. Our XHarvest implementation comprises two main parts, the firmware with 5K LOC and the enclave with 3K LOC. We develop the modified firmware in NVMeVirt, including the communication mechanism (§5.2), memory harvesting (§5.3), and the load detector (§5.4). The enclave part implements host CPU and memory harvesting (§5.1 and §5.3), communication mechanism (§5.2), and dynamic launch (§5.4). Note that XHarvest requires no modification to OS or applications, highlighting its practicality.

6 Evaluation

6.1 Experimental Setup

Methodology. We conduct our evaluation on a server equipped with dual Intel Xeon 8562Y+ CPU [41] and 480 GB DRAM. We set the read and write latency of the emulated SSD for 25us and 50us, respectively, aligning with high-performance SSDs [84, 132]. We also match its computing power to high-performance SSDs by

Workloads	lun0	lun2	lun6	Ali-1	Ali-2	Ali-3
Read ratio (%)	69.9	79.9	84.1	98.1	11	81.4
Avg. req size (KB)	21.9	21.8	24.9	36.6	29.6	374.1
Data size (GB)	48.7	53.1	99.4	32.9	28.2	61.6
Workloads	**casa**	**homes1**	**mds1**	**prn1**		**web0**
Read ratio (%)	0.5	9.6	92.9	75.3		29.9
Avg. req size (KB)	4	4.9	56.8	19.8		15.3
Data size (GB)	12.2	9.6	88.7	212.2		29.6

Table 2: The characteristics of examined workloads.

emulating the typical ARM processor embedded in these SSDs. The emulated firmware delivers 2000K IOPS under the 4K random pattern and saturates flash bandwidth with large sequential requests, aligning with the performance of SSD products [111, 113, 115]. Our NUMA-induced latency (75ns) setting aligns with the final latency targets of CXL specification [15]. We model the SSD energy consumption with McPAT [60] and DRAMPower [10], similar to prior works [30, 46, 120]. Table 1 lists our configurations.

Platforms. We compare XHarvest with existing SSD architectures. We also break down XHarvest to show the improvements brought by each component. (1) `ConvSSD`: the conventional SSD that integrates substantial internal resources. It runs firmware on the emulated ARM processor while maintaining a complete FTL mapping table in the internal memory. (2) `OCSSD`: the open-channel SSD that leverages host-side resources to execute the firmware and manage the metadata [8]. Since OCSSD has been removed from the Linux kernel, we emulate it on top of `ConvSSD` by running the firmware directly on the host CPU and residing its mapping table in the host memory. (3) `DLSSD`: a cost-efficient SSD called *DRAMless SSD* that omits internal DRAM and employs only hundreds of KB SRAM to buffer hot FTL metadata [45, 114, 125]. It builds an on-demand FTL cache like XHarvest to buffer metadata in host memory (typically 128MB for 1TB SSD [64]), but it needs to transfer the buffered translation pages from the host memory to the local SRAM via DMA for immediate access instead of accessing directly via CXL. To avoid firmware leakage, it encrypts the metadata buffered in the host FTL cache and decrypts those pages when transferring back [27, 100]. En/decryption is configured to take 3us per 4KB translation page, with a 5.1 GB/s overall bandwidth, aligning with current DRAMless SSDs [18, 23, 27, 44, 85]. (4) `DLSSD+LocalMem`: A variant of `DLSSD` with relaxed internal DRAM constraints for fair comparison with `Base`. (5) `Base`: the base SSD architecture of XHarvest, retaining only 25% of the internal compute power and 10% of the memory compared to `ConvSSD`. (6) `Base+CPU`: `Base` with host CPU harvesting. (7) `XHarvest`: a cost-efficient and high-performance SSD architecture that includes all our proposed designs, which stacks memory harvesting on `Base+CPU`. We restrict `XHarvest` to using only the same 128MB host memory as `DLSSD` for a fair comparison. Note that the existing prototypes validate the functionality and performance of TSP [104]. However, due to the lack of ready-to-integrate hardware, we have to overlook the marginal overhead of secure CXL traffic (cf. §5.2), slightly impacting the evaluation.

Workloads. We evaluate these SSD platforms with multiple block I/O workloads, including MSR, FIU, SYSTOR, and Alibaba block trace [3, 76, 97, 133]. These workloads cover read-intensive, write-intensive, and mixed scenarios with requests ranging from 4 to tens

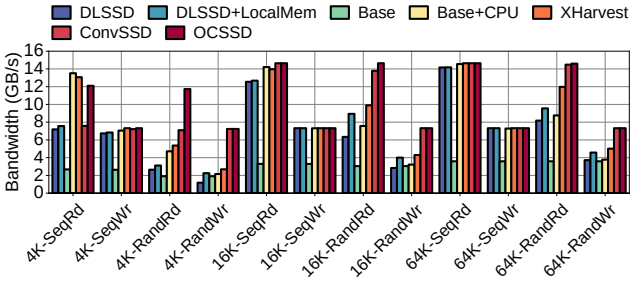


Figure 12: Bandwidth comparison in microbenchmark.

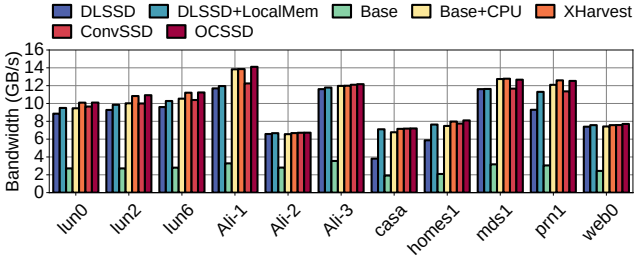


Figure 13: Bandwidth comparison in macrobenchmark.

of KB. By default, we use 16 I/O threads to generate asynchronous requests with 32 queue depth. Table 2 lists their key characteristics.

6.2 Overall Performance

Microbenchmark. Figure 12 shows the bandwidth comparison in microbenchmarks. `DLSSD` utilizes more powerful firmware to surpass `Base` by 1.22×. The additional internal memory in `DLSSD+LocalMem` enhances bandwidth by 38.19% in random workloads. In contrast, despite the same FTL cache capacity, `Base+CPU` surpasses `DLSSD` by 17.99% and 33.60% for sequential and random workloads, respectively. This benefit stems from the CXL interconnect, which aligns more effectively with the fine-grained mapping table access patterns as analyzed in §6.3. Moreover, it can safely harness the powerful host CPU via our CPU harvesting mechanism, augmenting its performance. `XHarvest` harvests the host-side memory to build a larger FTL cache, which in turn fully exploits the computing power of the host CPU. In `64K-RandRd`, it outperforms `Base+CPU` by 36.80%, whereas the same memory increment in `DLSSD+LocalMem` only results in 16.78% increase over `DLSSD`. `ConvSSD` eliminates the penalty of fetching translation pages by reserving sufficient memory to accommodate the complete mapping table, outperforming `Base+CPU` by 1.09× for random workloads. `XHarvest` narrows this gap to 63.04% by host memory harvesting. Note that the performance gains of `ConvSSD` derive from expensive computation and memory resource aggregation. Given that real workloads exhibit spatial locality, the gap is expected to diminish, as we discuss shortly. Furthermore, the harvested host CPU in `XHarvest` enables it to outperform `ConvSSD` in sequential workloads by 11.52%. `OCSSD`, which completely leverages host resources, outperforms `XHarvest` by 39.29%. However, its host resource reliance leads to severe contention with applications (cf. §3.2), as we analyze shortly.

Macrobenchmark. Figure 13 illustrates the bandwidth comparison in macrobenchmarks. `Base+CPU` and `XHarvest` improve the bandwidth by 17.39% and 6.31% over their respective counterparts

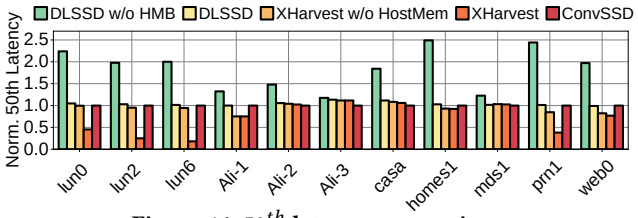
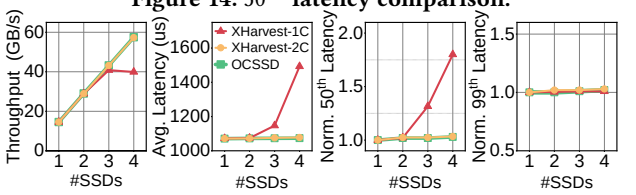
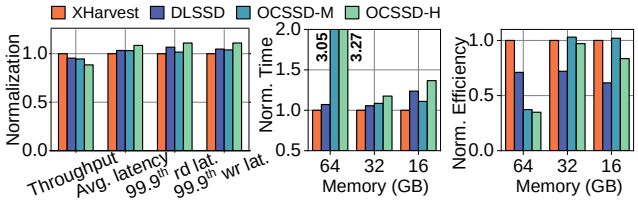


Figure 14: 50^{th} latency comparison.



(a) Throughtput. (b) Average latency. (c) 50^{th} latency. (d) 99^{th} latency.

Figure 15: Performance of multiple SSDs.



(a) Rocksdb. (b) Terasort execution time. (c) Cost efficiency.

Figure 16: Real-world application comparison.

in DRAMless SSD (i.e., `DLSSD` and `DLSSD+LocalMem`), respectively, owing to the CXL-enabled efficient FTL cache. Especially in random and small-size patterns (e.g., `casa`), `Base+CPU` outperforms `DLSSD` by 77.28% bandwidth. Compared to `ConvSSD`, `XHarvest` achieves a 5.02% bandwidth increase. This is because the spatial locality of real I/O traces mitigates the penalties of reading translation pages. `XHarvest` also reduces the bandwidth impact of these penalties by utilizing the powerful host CPU to exhaust the NAND flash parallelism. Note that we do not consider contention issues of `OCSSD` in macrobenchmarks, making it an ideal case. Although `XHarvest` introduces security overhead due to the enclave, it remains only 0.70% behind `OCSSD`. This minimal gap is attributed to CXL-driven efficient communication, which facilitates the seamless host-SSD collaboration. Our harmonized host-SSD coordination framework, including partitioning I/O loads and resources, effectively exploits both the SSD and the host.

50^{th} latency. Figure 14 compares the 50^{th} latency. We also compare this latency when SSDs cannot reserve host memory (i.e., `DLSSD w/o HMB` and `XHarvest w/o HostMem`). Note that frequent translation page reads/writes significantly impact the 50^{th} latency. `DLSSD` exhibits a 4.02% latency increase over `ConvSSD`. However, when host memory is scarce, `DLSSD w/o HMB` expands the increase to 83.28%. `XHarvest`, by exploiting the powerful host CPU and CXL, reduces 37.84% latency over `ConvSSD`. Only a minority of traces experience an increase (e.g., 11.65% rise in `Ali-3`). By reserving moderate internal memory, XHarvest also maintains performance stability even when host-side resources are insufficient, limiting the increase over `ConvSSD` (e.g., 8.37% in `casa`).

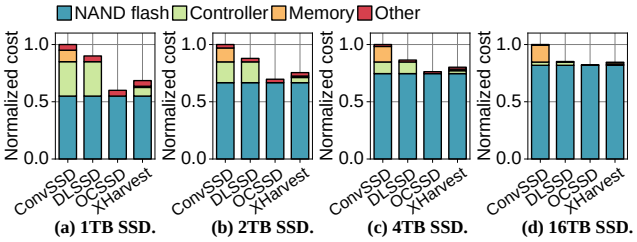
Multi-SSD. We use 64K random read to evaluate the scalability of multiple SSDs (cf. Figure 15). Note that SSDs incur negligible memory contention as XHarvest only harvests 128MB memory per

SSD, but they may cause CPU contention when they are served by the same enclave. We vary the number of enclaves (i.e., harvested CPU cores) in XHarvest to examine different CPU contention levels. To precisely measure this CPU contention impact, we exclude the overhead of translation page read/write by ensuring all I/O requests hit the FTL cache. We treat OCSSD as an ideal case by minimizing its potential CPU contention, which exhibits linear bandwidth scalability. XHarvest-2C (i.e., harvesting two CPU cores) also scales well compared to OCSSD, as enclaves on separate cores operate independently. With only one harvested core, CPU contention increases, but XHarvest-1C still saturates 3 SSDs, highlighting its efficiency. With 3 SSDs, it introduces 6.99% average latency and 31.51% median latency increase over OCSSD due to inadequate computing power. But this does not affect the tail latency, as it is determined by flash operations. Note that CPU resource constraints under high I/O loads are rare cases as analyzed in §3.3, highlighting the practicality of XHarvest. Considering that modern storage servers typically adopt 24 SSDs [19, 20, 102], XHarvest requires at most 8 CPU cores to saturate all SSDs, utilizing only 6.25% CPU resources of a 128-core storage server [20]. Moreover, peak load of all SSDs is rare to occur simultaneously (cf. §3.3), suggesting that XHarvest remains CPU-efficient under typical moderate loads.

Real-world application. We evaluate the impact of the host resource contention with a memory-intensive application (Rocksdb with mixgraph benchmark [29]), which executes 100 million operations on a 50GB dataset. We allocate 32GB memory for Rocksdb in XHarvest and DLSSD while constraining the memory in OCSSD to 90% and 80% of XHarvest to emulate medium (OCSSD-M) and high contention (OCSSD-H) due to its static and excessive memory occupancy. XHarvest induces minor CPU contention by harvesting only one CPU core to serve the SSD and incurs negligible memory contention as analyzed earlier. Figure 16a shows that XHarvest outperforms OCSSD-H by 11.59% in throughput and reduces average latency by 8.55%, benefiting from moderate resource harvesting. The heavy memory contention in OCSSD causes frequent page cache swapping and prolongs the request processing (e.g., 11.02% and 10.98% 99.9^{th} tail latency increases for read and write micro-operations in OCSSD-H). XHarvest surpasses DLSSD in throughput by 4.59% due to its efficient CXL-based FTL cache.

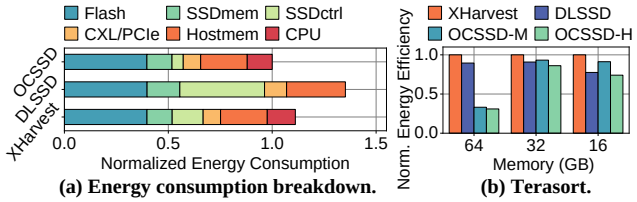
Resource contention analysis. We employ Terasort [37] to evaluate the performance stability, which generates sustained I/O bursts (e.g., 480s) to execute out-of-core sorting for a 35GB dataset. We vary the available application memory (i.e., 64, 32, and 16GB) and limit the memory in OCSSD (similar to Rocksdb setting) to generate different memory contention intensities. XHarvest harvests a single core from Terasort, incurring slight CPU contention. Note that as its working set (over 70GB) exceeds the available memory, Terasort frequently swaps data between memory and the SSD. Figure 16b shows that XHarvest outperforms DLSSD, OCSSD-M, and OCSSD-H across all contention levels, highlighting its performance stability. It reduces execution time by 2.27× and 36.71% with 64 and 16GB memory over OCSSD-H, respectively, due to reduced memory contention.

Cost analysis. Based on current market prices, we identify the cost of NAND flash at $5.248 per 128GB, memory at $7.5/GB, SSD controller at $23, and additional expenses (e.g., packaging) at $4 [50, 65, 70, 73, 74, 87, 108]. Note that SSDs typically equip 1GB memory per TB NAND flash for the FTL mapping table [93]. Figure 17 depicts the cost breakdown with varying capacity for all SSD architectures. For 1TB, 4TB, and 16TB SSD, XHarvest reduces the hardware cost by 31.50%, 19.83%, and 15.49%, respectively, while achieving the same or even higher performance than ConvSSD (cf. Figure 13). Compared to DLSSD, it integrates modest memory resources but reduces computation resources, saving 23.89% cost for 1TB SSD and increasing throughput by 17.39% in macrobenchmarks. Figure 16c details cost efficiency (i.e., performance per unit cost) in Terasort. XHarvest improves the cost efficiency by 19.21% and 28.15% over OCSSD-M and OCSSD-H, respectively. Note that as external interfaces and internal flash channel performance continuously increase (e.g., PCIe 6.0/7.0 and ONFi 5.2 [79, 94, 95]), the load on the SSD controller grows accordingly, significantly raising its costs and becoming a critical bottleneck in future high-performance SSDs.



(a) 1TB SSD. (b) 2TB SSD. (c) 4TB SSD. (d) 16TB SSD.

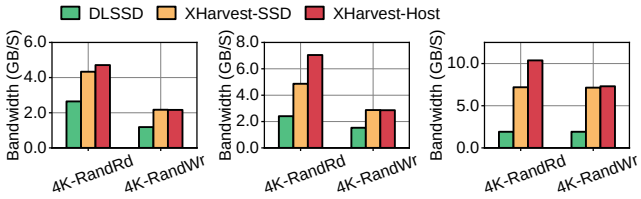
Figure 17: SSD cost with varying capacity.



(a) Energy consumption breakdown. (b) Terasort.

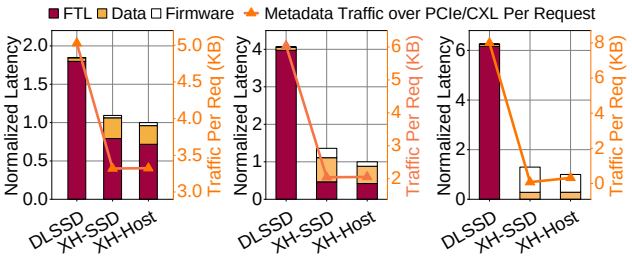
Figure 18: Energy consumption comparison.

Energy efficiency. Figure 18a shows the energy breakdown of storage-related hardware, including SSD and host resources, in prn1 macrobenchmark. OCSSD is an ideal case due to its lightweight design (e.g., omitting security and removing heavy SSD resource aggregations). Note that it requires a lightweight controller for flash operations and data movement, resulting in minor SSDctrl and SSDmem consumption. DLSSD incurs heavy metadata movement as analyzed before, increasing SSDmem, PCIe, and Hostmem by 30.19%, 26.62%, and 26.18%, respectively, over OCSSD. XHarvest exploits finer-grained CXL to reduce unnecessary metadata movements, incurring 0.34%, 0.06%, and 0.32% overhead for the same components compared to OCSSD. DLSSD relies on a powerful controller for I/O tasks (SSDctrl), incurring 1.34× more energy than OCSSD (SSDctrl and CPU), while XHarvest reduces this computation energy overhead (SSDctrl and CPU) to 63.04% via CXL-enabled simplified cache management. As a result, XHarvest incurs only 11.14% additional overall energy consumption over OCSSD. Figure 18b compares the system-level energy efficiency in Terasort. CPU and host memory used by applications dominate the overall consumption. OCSSD-H and DLSSD prolong application execution, incurring additional CPU and memory energy overhead and reducing energy efficiency by 36.29% and 14.07% over XHarvest.



(a) 25% hit ratio. (b) 50% hit ratio. (c) 100% hit ratio.

Figure 19: Throughput with different FTL cache hit ratio.

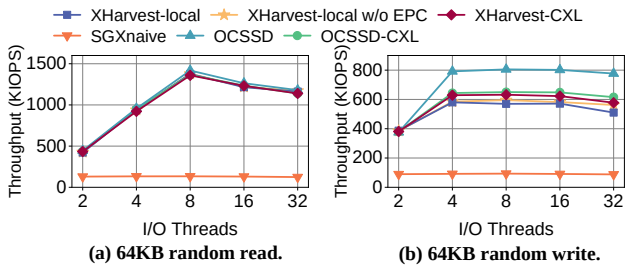


(a) 25% hit ratio. (b) 50% hit ratio. (c) 100% hit ratio.

Figure 20: Latency breakdown.

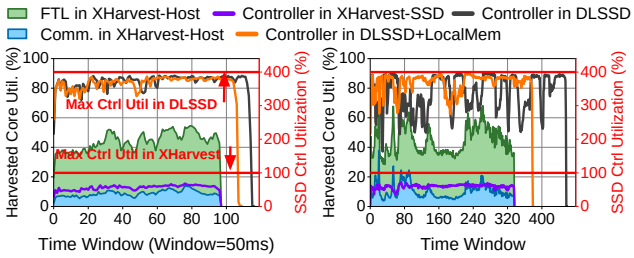
6.3 Performance Analysis

Memory harvesting. Figure 19 explores the benefits of our CXL-based host-side FTL cache. For a fair comparison, we remove the internal memory in XHarvest and it uses host memory. We also decompose XHarvest into `XHarvest-SSD`, which utilizes the internal computing power, and `XHarvest-Host` which relies on the host CPU. We relax the computing power constraint in `XHarvest-SSD` to prevent the bottlenecks. `XHarvest-SSD` and `XHarvest-Host` outperform `DLSSD` by 73.46% and 80.23% with 25% hit ratio, raising to 2.75× and 3.63× at 100% hit ratio. To delve into this enhancement, Figure 20 breaks down their latencies into `FTL` (en/decryption and translation page transfer), `Data` (actual data handling), and `Firmware` (firmware processing). We also analyze the average metadata traffic per request over CXL/PCIe, including FTL entries, communication mechanism, and AES-GCM encryption [14, 68, 105]. Note that only `XH-Host` requires communication, incurring 256B traffic per request (128B for sending a request to the host and 128B for translated address return). AES-GCM encryption only adds 12B MAC per transfer without altering plaintext size. Due to the separated coherency domains, `DLSSD` needs to move buffered translation pages to the SSD, resulting in `FTL` accounting for 97.28% in Figure 20a. Moreover, the DMA, optimized for bulk data transfers (e.g., 4KB), inefficiently handles finer mapping entries (i.e., 8B). This leads to unnecessary transfers (e.g., 8KB traffic per request in Figure 20c) and subsequent SRAM FTL cache pollution, amplifying the penalty. By utilizing CXL finer granularity, XHarvest diminishes these overheads. Consequently, at 25% hit ratio, `XH-SSD` and `XH-Host` cut traffic by 33.97% and 33.89% over `DLSSD`, resulting in `FTL` reduction by 55.95% and 60.16%. This latency reduction is amplified to 79.27% and 84.03% at 100% hit ratio. `XH-Host` benefits from the powerful CPU, diminishing `Firmware` (e.g., 22.93% over `XH-SSD` in Figure 20c). Note that the communication mechanism and encryption overheads in `XH-Host` are minor (e.g., contributing 9.19% metadata traffic in Figure 20a) and also negligible compared to real data transfers (e.g. increasing transferred data volume by only 7.64% and 0.47% for 4KB and 64KB request).



(a) 64KB random read. (b) 64KB random write.

Figure 21: Firmware performance comparison.



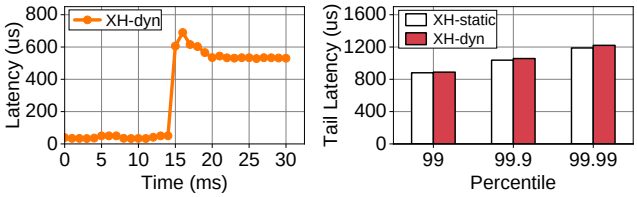
(a) lun2. (b) prn1.

Figure 22: Computation resource usage.

Host CPU harvesting. Figure 21 shows the throughput of the enclave-based firmware using the host-side EPC and the SSD internal memory (`XHarvest-local` and `XHarvest-CXL`). For finer comparison, we evaluate their non-protected counterpart, `OCSSD` and `OCSSD-CXL`, which use the same memory configurations but without SGX protection. Note that we focus solely on the firmware processing capability, excluding NAND flash from the emulation model. `SGXnaive` directly imports the firmware into the enclave, resulting in inefficient host-SSD interaction through ecall/ocall and the complex OS stack. `XHarvest-CXL` utilizes the CXL-driven communication mechanism to address this bottleneck, outperforming `SGXnaive` by 6.85×. Due to the simple FTL tasks, `XHarvest-CXL` maintains only 3.80% throughput gap from `OCSSD-CXL`, showing the low enclave execution overhead. It degrades by 25.66% in random write compared to `OCSSD` due to CXL-related penalties. While `XHarvest-local` incurs encryption when exchanging FTL entries between the EPC and the CPU LLC, its simple EPC access pattern (only one FTL entry per I/O request) mitigates this penalty. It exhibits only 5.09% decrease in `64KB random write` over `XHarvest-local w/o EPC` (i.e., enclave builds FTL cache in regular memory instead of EPC). This overhead does not affect `64KB random read` due to simpler read request processing.

Computation resource usage. Figure 22 details the computation resource usage of the SSD controller and the single harvested CPU core. To capitalize on the extensive flash parallelism, `DLSSD` and `DLSSD+LocalMem` nearly exhaust all computation resources. However, this high utilization is temporary (cf. §3.2), leading to frequent idle periods. `XHarvest` addresses this inefficiency by dynamic host CPU harvesting, removing underutilized resources and their costs. Moreover, our harmonized host-SSD collaboration balances computation loads across the enclave and SSD controller well, harnessing their computing power. Note that the single harvested core remains available to serve more SSDs.

Dynamic enclave launch. Figure 23 illustrates the impact of dynamic enclave launch on real-time and tail latency during a shift



(a) Real-time latency. (b) Tail latency.

Figure 23: Dynamic enclave launch analysis.

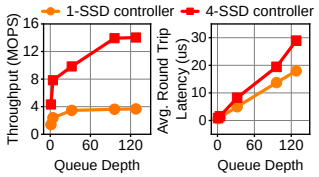


Figure 24: Communication mechanism analysis.

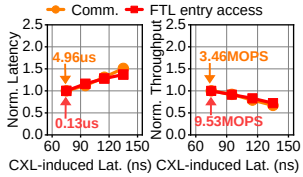


Figure 25: Sensitivity of CXL latency.

from the low load (1 thread with 1 depth) to the high load (4 threads with 32 depths). By separating the enclave initialization from the critical path, the SSD overload caused by waiting for enclave launch is limited to 5ms. Note that the load monitor may signal prematurely if the cumulative load exceeds the threshold for that period. This transient increase in real-time latency has a negligible impact on the tail latency (cf. Figure 23b) over longer durations (e.g., 60s).

Communication mechanism. We run the loopback microbenchmark to evaluate the standalone communication mechanism (cf. Figure 24). Each SSD controller sends requests via its dedicated message queue (cf. §5.2), while the enclave directly forwards the received requests back via another queue. With a single SSD controller, this mechanism delivers 3700KOPS throughput, easily meeting the need of high-performance SSDs (e.g., 2000KOPS throughput). It achieves low latency (e.g., near 1us at 1 queue depth), while also maintaining scalability (e.g., 3.85× throughput increase when scaling to four SSD controllers). Note that the polling enables high-performance host-SSD communication required by high-performance SSDs, which traditional interrupt struggles to achieve [52, 123]. The polling also does not compromise CPU efficiency, since only high loads activate this mechanism and these loads inherently fully utilize the CPU core for FTL tasks and communication.

Sensitivity of CXL latency. We use the cycle-level simulator *Sniper* [24, 36] to evaluate the sensitivity of our CXL-related designs (i.e., CXL-based communication mechanism and CXL-based FTL cache) by varying CXL-induced latencies. Figure 25 shows that our CXL designs maintain stable performance across different CXL-induced latencies, which incurs linear performance penalties. For example, as the CXL-induced latency increases from 75ns to 135ns, the latency of the communication (32 depth) increases by 51.16% while throughput decreases by 32.93%. However, this throughput reduction does not introduce a bottleneck, as it well satisfies the SSD requirement. Similarly, the latency of sequential FTL entry access increases by 37.13% while throughput decreases by 27.85%. Furthermore, these incurred us-level penalties (e.g., Figure 24) impact overall performance negligibly, as it is dominated by FTL computation and flash operations (e.g., 50us write latency). Notably, the CXL-induced latency setting in XHarvest (75ns) matches the final latency target in CXL Specification [15], showing its expected performance on future mature CXL hardware.

7 Related Work and Discussion

Cost-efficient storage. Zoned Namespace (ZNS) SSD represents a new cost-efficient SSD architecture [7, 75, 127]. It removes the internal memory and constructs a smaller host-side mapping table by organizing flash blocks into coarse-grained zones. However, its append-only write constraint limits the usability and broader adoption [127]. Unlike ZNS SSDs, XHarvest focuses on mainstream block-interface SSDs and allows user applications to benefit from our design without modifications. LMB [119] explores CXL to replace DMA to improve the host-based FTL cache in DRAMless SSDs, but it inherits the fixed memory reservation and overlooks computational resources within these SSDs. BlockFlex [86] harvests underutilized flash resources in cloud environments to maximize capacity utilization, whereas XHarvest targets underutilized computational and memory resources. It pioneers cost-efficient SSD designs via dynamically harvesting host resources, a new strategy unexplored before.

CXL. The emerging CXL interconnect has attracted considerable attention in recent research. Pond [57] establishes a CXL-based shared memory pool for virtual machines, effectively reducing DRAM-related costs. CXL-SSDs [42, 53, 88, 126] combine flash memory and DRAM by leveraging CXL as a new data path in SSDs, thereby developing high-capacity and high-performance storage class memory (SCM) devices. XHarvest, while sharing similar cost-efficient and high-performance goals, exploits CXL as a new SSD control path (i.e., delegating the control path to the host). This new path exploration enables host-SSD coordination for I/O task, thus reducing reliance on SSD internal resources.

TEE. Beyond Intel SGX, AMD SEV, ARM TrustZone, and RISC-V Keystone provide similar security features [44, 58, 103]. Porting XHarvest to these platforms is possible with the necessary adaptation, such as migrating to the appropriate SDK [103]. TEEs have been widely adopted for data security, with prior works leveraging them to protect databases [4, 48, 67] and using TrustZone to secure in-storage computing [44]. Note that these efforts primarily focus on data security. XHarvest uniquely integrates TEE with a new CXL feature (i.e., TSP) to secure host-based SSD firmware execution and protect host-SSD communication, enabling a new computing paradigm (i.e., secure computation delegating from devices to host).

XHarvest practicality. Driven by hardware cost consideration, which is one of the most critical factors in SSD designs, vendors have embraced host-SSD coordination for cost-efficient SSD designs (e.g., DRAMless SSD products [114]). XHarvest takes this trend one step further. It harvests both memory and computation resources, reducing 31.5% cost without a performance penalty. Importantly, while XHarvest investigates future SSD designs, it is built on commercially deployed technologies (CXL and TEE) rather than proprietary mechanisms, ensuring its industry compatibility. By adhering to standardized technologies, XHarvest requires only firmware modification, avoiding hardware redesign. Aligning with the growing adoption of CXL and TEE (e.g., CXL/TEE-enabled CPUs and CXL-SSDs [39, 40, 88]), we believe its deployment is both practical and viable.

8 Conclusion

In this work, we propose XHarvest, which harnesses CXL and SGX to facilitate dynamic, efficient, and secure host resource harvesting. XHarvest employs TEE to shield the host-based firmware execution while leveraging the CXL to build a unified and efficient metadata cache. Our evaluation results reveal that XHarvest achieves 31.50% cost savings and 5.02% higher throughput over conventional SSDs.

Acknowledgments

We sincerely thank the anonymous reviewers for their insightful feedback. This work is mainly supported by the Natural Science Foundation of China under Grant No. 62332021, 62472007, and 624B2004. Dr. Chen is partially supported by the Natural Science Foundation of China (Project No.62372073), the Fundamental Research Funds for the Central Universities under Grant (Project No.2023CDJXY-039), and Chongqing Post-doctoral Science Foundation (Project No.2021LY75 and cx2023080). Dr. Liang is partially supported by the Natural Science Foundation of China under Grant No. 62202453. The corresponding author is Jie Zhang.

References

[1] Mohammadamin Ajdari, Pyeongsu Park, Joonsung Kim, Dongup Kwon, and Jangwoo Kim. 2019. CIDR: A cost-effective in-line data reduction system for terabit-per-second scale SSD arrays. In *2019 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 28–41.

[2] Alibaba. 2018. Alibaba Open Cluster Trace Program. https://github.com/alibaba/clusterdata/blob/master/cluster-trace-v2018/trace_2018.md.

[3] Alibaba. 2024. Alibaba Block Traces. https://github.com/alibaba/block-traces.

[4] Maurice Bailleu, Jörg Thalheim, Pramod Bhatotia, Christof Fetzer, Michio Honda, and Kapil Vaswani. 2019. {SPEICHER}: Securing {LSM-based}{Key-Value} Stores using Shielded Execution. In *17th USENIX Conference on File and Storage Technologies (FAST 19)*. 173–190.

[5] Erick Bauman, Huibo Wang, Mingwei Zhang, and Zhiqiang Lin. 2018. Sgxelide: enabling enclave code secrecy via self-modification. In *Proceedings of the 2018 International Symposium on Code Generation and Optimization*. 75–86.

[6] Bharat Bhushan. 2023. Current status and outlook of magnetic data storage devices. *Microsystem Technologies* 29, 11 (2023), 1529–1546.

[7] Matias Bjørling, Abutalib Aghayev, Hans Holmberg, Aravind Ramesh, Damien Le Moal, Gregory R Ganger, and George Amvrosiadis. 2021. {ZNS}: Avoiding the block interface tax for flash-based {SSDs}. In *2021 USENIX Annual Technical Conference (USENIX ATC 21)*. 689–703.

[8] Matias Bjørling, Javier Gonzalez, and Philippe Bonnet. 2017. {LightNVM}: The linux {Open-Channel}{SSD} subsystem. In *15th USENIX Conference on File and Storage Technologies (FAST 17)*. 359–374.

[9] Zhichao Cao. 2020. *High-Performance and Cost-Effective Storage Systems for Supporting Big Data Applications*. Ph. D. Dissertation. University of Minnesota.

[10] Karthik Chandrasekar, Christian Weis, Yonghui Li, Benny Akesson, Norbert Wehn, and Kees Goossens. 2012. DRAMPower: Open-source DRAM power & energy estimation tool. *URL: http://www. drampower. info* 22 (2012).

[11] Feng Chen, Tian Luo, and Xiaodong Zhang. 2011. {CAFTL}: A {Content-Aware} flash translation layer enhancing the lifespan of flash memory based solid state drives. In *9th USENIX Conference on File and Storage Technologies (FAST 11)*.

[12] Guoxing Chen, Yinqian Zhang, and Ten-Hwang Lai. 2019. Opera: Open remote attestation for intel's secure enclaves. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*. 2317–2331.

[13] Huili Chen, Cheng Fu, Bita Darvish Rouhani, Jishen Zhao, and Farinaz Koushanfar. 2019. DeepAttest: An end-to-end attestation framework for deep neural networks. In *Proceedings of the 46th International Symposium on Computer Architecture*. 487–498.

[14] Kwanghoon Choi, Igjae Kim, Sunho Lee, and Jaehyuk Huh. 2024. ShieldCXL: A Practical Obliviousness Support with Sealed CXL Memory. *ACM Transactions on Architecture and Code Optimization* (2024).

[15] CXL. 2024. Compute Express Link. https://computeexpresslink.org/.

[16] CXL. 2024. Introducing the CXL 3.1 Specification. https://computeexpresslink.org/wp-content/uploads/2024/03/CXL_3.1-Webinar-Presentation_Feb_2024.pdf.

[17] Dapustor. 2022. DapuStor Roalsen SSD R5100 7.5 TB. https://www.techpowerup.com/ssd-specs/dapustor-roalsen-ssd-r5100-7-5-tb.d2169.

[18] Delkin. 2024. Encryption and Security Development in Solid State Storage Devices (SSD). https://www.delkin.com/blog/encryption-and-security-development-in-solid-state-storage-devices-ssd/.

[19] Dell. 2023. Dell PowerEdge R740xd. https://i.dell.com/sites/csdocuments/Shared-Content_data-Sheets_Documents/en/poweredge-r740xd-spec-sheet.pdf.

[20] Dell. 2023. Dell PowerEdge R7615. https://www.delltechnologies.com/asset/pl-pl/products/servers/technical-support/poweredge-r7615-spec-sheet.pdf.

[21] Li Deng, Yu-Lin Ren, Fei Xu, Heng He, and Chao Li. 2018. Resource utilization analysis of Alibaba cloud. In *Intelligent Computing Theories and Application: 14th International Conference, ICIC 2018, Wuhan, China, August 15-18, 2018, Proceedings, Part I 14*. Springer, 183–194.

[22] ARM developer. 2024. Cortex-R CPU performance comparison. https://developer.arm.com/compare-ip/#cortex-r-cpu-performance---scalar.

[23] Jaeyoung Do, Victor C Ferreira, Hossein Bobarshad, Mahdi Torabzadehkashi, Siavash Rezaei, Ali Heydarigorji, Diego Souza, Brunno F Goldstein, Leandro Santiago, Min Soo Kim, et al. 2020. Cost-effective, energy-efficient, and scalable storage computing for large-scale AI applications. *ACM Transactions on Storage (TOS)* 16, 4 (2020), 1–37.

[24] Juechu Dong, Jonah Rosenblum, and Satish Narayanasamy. 2024. Toleo: Scaling Freshness to Tera-scale Memory using CXL and PIM. *arXiv preprint arXiv:2410.12749* (2024).

[25] DRAMeXchange. 2024. The Global Price of NAND Flash and LPDDR. https://www.dramexchange.com/.

[26] Nir Drucker, Shay Gueron, and Vlad Krasnov. 2018. Making AES great again: the forthcoming vectorized AES instruction. Cryptology ePrint Archive, Paper 2018/392. https://eprint.iacr.org/2018/392

[27] Lingyan Fan, Jianjun Luo, Hailuan Liu, and Xuan Geng. 2014. Data security concurrent with homogeneous by AES algorithm in SSD controller. *IEICE Electronics Express* 11, 13 (2014), 20140535–20140535.

[28] Erhu Feng, Dahu Feng, Dong Du, Yubin Xia, Wenbin Zheng, Siqi Zhao, and Haibo Chen. 2024. sIOPMP: Scalable and Efficient I/O Protection for TEEs. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. 1061–1076.

[29] Guanyu Feng, Huanqi Cao, Xiaowei Zhu, Bowen Yu, Yuanwei Wang, Zixuan Ma, Shengqi Chen, and Wenguang Chen. 2022. {TriCache}: A {User-Transparent} Block Cache Enabling {High-Performance}{Out-of-Core} Processing with {In-Memory} Programs. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. 395–411.

[30] Donghyun Gouk, Miryeong Kwon, Jie Zhang, Sungjoon Koh, Wonil Choi, Nam Sung Kim, Mahmut Kandemir, and Myoungsoo Jung. 2018. Amber: Enabling precise full-system simulation with detailed modeling of all SSD resources. In *2018 51st Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 469–481.

[31] Théophile Gousselot, Jean-Max Dutertre, Olivier Potin, and Jean-Baptiste Rigaud. 2025. Code Encryption for Confidentiality and Execution Integrity down to Control Signals. In *IEEE International Symposium on Hardware Oriented Security and Trust (HOST)*.

[32] Jing Guo, Zihao Chang, Sa Wang, Haiyang Ding, Yihui Feng, Liang Mao, and Yungang Bao. 2019. Who limits the resource efficiency of my datacenter: An analysis of alibaba datacenter traces. In *Proceedings of the international symposium on quality of service*. 1–10.

[33] Yunyan Guo and Guoliang Li. 2024. A CXL-Powered Database System: Opportunities and Challenges. In *2024 IEEE 40th International Conference on Data Engineering (ICDE)*. IEEE, 5593–5604.

[34] Gabriel Haas and Viktor Leis. 2023. What modern NVMe storage can do, and how to exploit it: high-performance I/O for high-performance storage engines. *Proceedings of the VLDB Endowment* 16, 9 (2023), 2090–2102.

[35] Xiaobin He, Bin Yang, Jie Gao, Wei Xiao, Qi Chen, Shupeng Shi, Dexun Chen, Weiguo Liu, Wei Xue, and Zuo-ning Chen. 2023. {HadaFS}: A File System Bridging the Local and Shared Burst Buffer for Exascale Supercomputers. In *21st USENIX Conference on File and Storage Technologies (FAST 23)*. 215–230.

[36] Wim Heirman, Trevor Carlson, and Lieven Eeckhout. 2012. Sniper: Scalable and accurate parallel multi-core simulation. In *8th International Summer School on Advanced Computer Architecture and Compilation for High-Performance and Embedded Systems (ACACES-2012)*. High-Performance and Embedded Architecture and Compilation Network of ..., 91–94.

[37] Shengsheng Huang, Jie Huang, Jinquan Dai, Tao Xie, and Bo Huang. 2010. The HiBench benchmark suite: Characterization of the MapReduce-based data analysis. In *2010 IEEE 26th International conference on data engineering workshops (ICDEW 2010)*. IEEE, 41–51.

[38] Intel. 2024. Intel(R) Software Guard Extensions for Linux* OS. https://github.com/intel/linux-sgx.

[39] Intel. 2024. Intel® Processors Supporting Intel® SGX. https://www.intel.com/content/www/us/en/architecture-and-technology/software-guard-extensions-processors.html.

[40] Intel. 2024. Intel® Xeon® 6 Processors with Efficient-Cores (E-Cores). https://www.intel.com/content/www/us/en/products/details/processors/

xeon/xeon6-e-cores.html.
[41] Intel. 2024. Intel® Xeon® Platinum 8562Y+ Processor. https://www.intel.com/content/www/us/en/products/sku/237558/intel-xeon-platinum-8562y-processor-60m-cache-2-80-ghz/specifications.html.
[42] Myoungsoo Jung. 2022. Hello bytes, bye blocks: Pcie storage meets compute express link for memory expansion (cxl-ssd). In *Proceedings of the 14th ACM Workshop on Hot Topics in Storage and File Systems*. 45–51.
[43] Panos Kampanakis, Matt Campagna, Eric Crocket, Adam Petcher, and Shay Gueron. 2023. Practical challenges with AES-GCM and the need for a new cipher. In *Third NIST Workshop on Block Cipher Modes of Operation*.
[44] Luyi Kang, Yuqi Xue, Weiwei Jia, Xiaohao Wang, Jongryool Kim, Changhwan Youn, Myeong Joon Kang, Hyung Jin Lim, Bruce Jacob, and Jian Huang. 2021. Iceclave: A trusted execution environment for in-storage computing. In *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*. 199–211.
[45] Hyukjoong Kim, Dongkun Shin, Yun Ho Jeong, and Kyung Ho Kim. 2017. {SHRD}: Improving Spatial Locality in Flash Storage Accesses by Sequentializing in Host and Randomizing in Device. In *15th USENIX Conference on File and Storage Technologies (FAST 17)*. 271–284.
[46] Junkyum Kim, Myeonggu Kang, Yunki Han, Yang-Gon Kim, and Lee-Sup Kim. 2023. Optimstore: In-storage optimization of large scale dnns with on-die processing. In *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 611–623.
[47] Sang-Hoon Kim, Jaehoon Shim, Euidong Lee, Seongyeop Jeong, Ilkueon Kang, and Jin-Soo Kim. 2023. {NVMeVirt}: A Versatile Software-defined Virtual {NVMe} Device. In *21st USENIX Conference on File and Storage Technologies (FAST 23)*. 379–394.
[48] Taehoon Kim, Joongun Park, Jaewook Woo, Seungheun Jeon, and Jaehyuk Huh. 2019. Shieldstore: Shielded in-memory key-value storage with sgx. In *Proceedings of the Fourteenth EuroSys Conference 2019*. 1–15.
[49] Yoona Kim, Inhyuk Choi, Juhyung Park, Jaeheon Lee, Sungjin Lee, and Jihong Kim. 2023. Integrated {Host-SSD} Mapping Table Management for Improving User Experience of Smartphones. In *21st USENIX Conference on File and Storage Technologies (FAST 23)*. 441–456.
[50] Kingston. 2024. Kingston Q6422PM3BDGVK-U. https://www.mouser.hk/ProductDetail/Kingston/Q6422PM3BDGVK-U?qs=iLKYxzqNS75joQ5Bm1ptMg%3D%3D.
[51] Ana Klimovic, Christos Kozyrakis, Eno Thereska, Binu John, and Sanjeev Kumar. 2016. Flash storage disaggregation. In *Proceedings of the Eleventh European Conference on Computer Systems*. 1–15.
[52] Sungjoon Koh, Junhyeok Jang, Changrim Lee, Miryeong Kwon, Jie Zhang, and Myoungsoo Jung. 2019. Faster than flash: An in-depth study of system challenges for emerging ultra-low latency SSDs. In *2019 IEEE International Symposium on Workload Characterization (IISWC)*. IEEE, 216–227.
[53] Miryeong Kwon, Sangwon Lee, and Myoungsoo Jung. 2023. Cache in hand: Expander-driven cxl prefetcher for next generation cxl-ssd. In *Proceedings of the 15th ACM Workshop on Hot Topics in Storage and File Systems*. 24–30.
[54] Gyusun Lee, Seokha Shin, Wonsuk Song, Tae Jun Ham, Jae W Lee, and Jinkyu Jeong. 2019. Asynchronous {I/O} stack: A low-latency kernel {I/O} stack for {Ultra-Low} latency {SSDs}. In *2019 USENIX Annual Technical Conference (USENIX ATC 19)*. 603–616.
[55] Sungjin Lee, Ming Liu, Sangwoo Jun, Shuotao Xu, Jihong Kim, et al. 2016. {Application-Managed} Flash. In *14th USENIX Conference on File and Storage Technologies (FAST 16)*. 339–353.
[56] Alberto Lerner and Gustavo Alonso. 2024. CXL and the Return of Scale-Up Database Engines. *arXiv preprint arXiv:2401.01150* (2024).
[57] Huaicheng Li, Daniel S Berger, Lisa Hsu, Daniel Ernst, Pantea Zardoshti, Stanko Novakovic, Monish Shah, Samir Rajadnya, Scott Lee, Ishwar Agarwal, et al. 2023. Pond: Cxl-based memory pooling systems for cloud platforms. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. 574–587.
[58] Mengyuan Li, Yinqian Zhang, Zhiqiang Lin, and Yan Solihin. 2019. Exploiting unprotected {I/O} operations in {AMD's} secure encrypted virtualization. In *28th USENIX Security Symposium (USENIX Security 19)*. 1257–1272.
[59] Qiang Li, Qiao Xiang, Yuxin Wang, Haohao Song, Ridi Wen, Wenhui Yao, Yuanyuan Dong, Shuqi Zhao, Shuo Huang, Zhaosheng Zhu, et al. 2023. More than capacity: Performance-oriented evolution of pangu in alibaba. In *21st USENIX Conference on File and Storage Technologies (FAST 23)*. 331–346.
[60] Sheng Li, Jung Ho Ahn, Richard D Strong, Jay B Brockman, Dean M Tullsen, and Norman P Jouppi. 2009. McPAT: An integrated power, area, and timing modeling framework for multicore and manycore architectures. In *Proceedings of the 42nd annual ieee/acm international symposium on microarchitecture*. 469–480.
[61] Shaobo Li, Yirui Zhou, Hao Ren, and Jian Huang. 2025. ByteFS: System Support for (CXL-based) Memory-Semantic Solid-State Drives. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*. 116–132.
[62] Shih-Wei Li, John S Koh, and Jason Nieh. 2019. Protecting cloud virtual machines from hypervisor and host operating system exploits. In *28th USENIX Security Symposium (USENIX Security 19)*. 1357–1374.
[63] Linux. 2021. deprecated OCSSD support and schedule it for removal in Linux 5.15. https://patchwork.kernel.org/project/linux-block/patch/20210412081257.2585860-1-hch@lst.de/#24107773.
[64] Linux. 2024. Linux NVMe driver. https://github.com/torvalds/linux/tree/master/drivers/nvme.
[65] Longsys. 2021. Application Documents for Initial Public Offering and Listing on GEM of Shenzhen Jiang Bolong Electronics Co. Initial Public Offering of Shares and Listing on GEM Board Application Documents Report in response to the audit inquiry letter from. https://qccdata.qichacha.com/ReportData/PDF/ef39602f078481d5136f3c4253bc56b5.pdf.
[66] LPC. 2022. CXL Confidential Computing. https://lpc.events/event/16/contributions/1250/attachments/.
[67] Chengyan Ma, Di Lu, Chaoyue Lv, Ning Xi, Xiaohong Jiang, Yulong Shen, and Jianfeng Ma. 2024. BiTDB: Constructing A Built-in TEE Secure Database for Embedded Systems. *IEEE Transactions on Knowledge and Data Engineering* (2024).
[68] Raghu Makaram and David Harriman. 2023. Compute Express Link™ (CXL™) Link-level Integrity and Data Encryption (CXL IDE). https://computeexpresslink.org/wp-content/uploads/2023/12/CXL_CXL-IDE-Webinar_FINAL.pdf.
[69] Bo Mao, Suzhen Wu, and Lide Duan. 2017. Improving the SSD performance by exploiting request characteristics and internal parallelism. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 37, 2 (2017), 472–484.
[70] China Flash Market. 2024. The Price of NAND Flash and LPDDR. https://en.chinaflashmarket.com/.
[71] Marvell. 2022. Bravera SC5 SSD Controllers. https://www.marvell.com/products/ssd-controllers/mv-ss1331-1333.html.
[72] Rino Micheloni, Luca Crippa, and Alessia Marelli. 2010. *Inside NAND flash memories*. Springer Science & Business Media.
[73] Micron. 2024. Micron MT53E512M32D1ZW-046BAIT:B. https://www.mouser.hk/ProductDetail/Micron/MT53E512M32D1ZW-046BAITB?qs=3Rah4i%252BhyCEfB4XYGpjn0A%3D%3D.
[74] Micron. 2024. Micron MT62F512M32D2DS-031. https://www.mouser.com/ProductDetail/Micron/MT62F512M32D2DS-031-ITB-TR?qs=pUKx8fyJudAdsuj8e0%2F53A%3D%3D.
[75] Jaehong Min, Chenxingyu Zhao, Ming Liu, and Arvind Krishnamurthy. 2023. {eZNS}: An elastic zoned namespace for commodity {ZNS}{SSDs}. In *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23)*. 461–477.
[76] Storage networking industry association. 2024. the snia's i/o traces, tools, and analysis (iotta) repository. http://iotta.snia.org/.
[77] NVMe. 2024. NVM Express® (NVMe®) specifications. https://nvmexpress.org/specifications/.
[78] Hyunyoung Oh, Adil Ahmad, Seonghyun Park, Byoungyoung Lee, and Yunheung Paek. 2020. Trustore: Side-channel resistant storage for sgx using intel hybrid cpu-fpga. In *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*. 1903–1918.
[79] ONFi. 2024. ONFI 5.2 Specifications. https://onfi.org/specs.html.
[80] ONFi. 2024. Open NAND Flash Interface Specifications. https://onfi.org/specs.html.
[81] Dahan Pan, Jianqiang Wang, Yingpeng Chen, Donghui Yu, Wenbo Yang, and Yuanyuan Zhang. 2024. COMURICE: Closing Source Code Leakage in Cloud-Based Compiling via Enclave. In *2024 IEEE 11th International Conference on Cyber Security and Cloud Computing (CSCloud)*. IEEE, 126–131.
[82] Ivy Peng, Ian Karlin, Maya Gokhale, Kathleen Shoga, Matthew Legendre, and Todd Gamblin. 2021. A holistic view of memory utilization on HPC systems: Current and future trends. In *Proceedings of the International Symposium on Memory Systems*. 1–11.
[83] Hongwei Qin, Dan Feng, Wei Tong, Jingning Liu, and Yutong Zhao. 2019. Qblk: Towards fully exploiting the parallelism of open-channel ssds. In *2019 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 1064–1069.
[84] Ziyue Qiu, Juncheng Yang, Juncheng Zhang, Cheng Li, Xiaosong Ma, Qi Chen, Mao Yang, and Yinlong Xu. 2023. Frozenhot cache: Rethinking cache management for modern hardware. In *Proceedings of the Eighteenth European Conference on Computer Systems*. 557–573.
[85] Benjamin Reidys, Peng Liu, and Jian Huang. 2022. Rssd: Defend against ransomware with hardware-isolated network-storage codesign and post-attack analysis. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*. 726–739.
[86] Benjamin Reidys, Jinghan Sun, Anirudh Badam, Shadi Noghabi, and Jian Huang. 2022. {BlockFlex}: Enabling Storage Harvesting with {Software-Defined} Flash in Modern Cloud Platforms. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. 17–33.
[87] Ryan S. 2020. SSD cost calculator sources. https://www.soothsawyer.com/wp-content/uploads/2020/03/Public_SSD_Cost_Calculator_Share.pdf.
[88] Samsung. 2024. Industry's 1st CXL-based Storage Optimized for AI/ML. https://semiconductor.samsung.com/us/about-us/us-office/us-r-and-d-

labs/memory-labs/cxl-memory-module-hybrid/.
[89] Felix Schuster, Manuel Costa, Cédric Fournet, Christos Gkantsidis, Marcus Peinado, Gloria Mainar-Ruiz, and Mark Russinovich. 2015. VC3: Trustworthy data analytics in the cloud using SGX. In *2015 IEEE symposium on security and privacy*. IEEE, 38–54.
[90] Yizhou Shan, Yutong Huang, Yilun Chen, and Yiying Zhang. 2018. {LegoOS}: A disseminated, distributed {OS} for hardware resource disaggregation. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*. 69–87.
[91] Xuanhua Shi, Ming Li, Wei Liu, Hai Jin, Chen Yu, and Yong Chen. 2017. Ssdup: a traffic-aware ssd burst buffer for hpc systems. In *Proceedings of the international conference on supercomputing*. 1–10.
[92] Xuanhua Shi, Wei Liu, Ligang He, Hai Jin, Ming Li, and Yong Chen. 2020. Optimizing the SSD burst buffer by traffic detection. *ACM Transactions on Architecture and Code Optimization (TACO)* 17, 1 (2020), 1–26.
[93] Ji-Yong Shin, Zeng-Lin Xia, Ning-Yi Xu, Rui Gao, Xiong-Fei Cai, Seungryoul Maeng, and Feng-Hsiung Hsu. 2009. FTL design exploration in reconfigurable high-performance SSD for server applications. In *Proceedings of the 23rd international conference on Supercomputing*. 338–349.
[94] PCI SIG. 2024. PCI Express 6.0 Specification. https://pcisig.com/pci-express-6.0-specification.
[95] PCI SIG. 2024. PCIe® 7.0 Specification, Version 0.5 Now Available: Full Draft Available to Members. https://pcisig.com/blog/pcie%C2%AE-70-specification-version-05-now-available-full-draft-available-members.
[96] Joonseop Sim, Soohong Ahn, Taeyoung Ahn, Seungyong Lee, Myunghyun Rhee, Jooyoung Kim, Kwangsik Shin, Donguk Moon, Euiseok Kim, and Kyoung Park. 2024. Computational CXL-Memory Solution for Accelerating Memory-Intensive Applications. In *2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 615–615.
[97] SNIA. 2024. Fiu traces. http://iotta.snia.org/traces/block-io/390.
[98] Yaroslav Sovyn, Volodymyr Khoma, and Michal Podpora. 2019. Comparison of three CPU-core families for IoT applications in terms of security and performance of AES-GCM. *IEEE Internet of Things Journal* 7, 1 (2019), 339–348.
[99] Avinash Srinivasan, Jie Wu, Panneer Santhalingam, and Jeffrey Zamanski. 2014. Deaddrop-in-a-flash: Information hiding at SSD nand flash memory physical layer. *SECURWARE* 79 (2014), 2014.
[100] Flash Memory Summit. 2017. Improving the Design of DRAM-Less PCIe SSD. https://files.futurememorystorage.com/proceedings/2017/20170808_S101A_Yang.pdf.
[101] Yan Sun, Yifan Yuan, Zeduo Yu, Reese Kuper, Chihun Song, Jinghan Huang, Houxiang Ji, Siddharth Agarwal, Jiaqi Lou, Ipoom Jeong, et al. 2023. Demystifying cxl memory with genuine cxl-ready systems and devices. In *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*. 105–121.
[102] Supermicro. 2025. Storage SuperServer SSG-229J-5BU24JBF. https://www.supermicro.com/zh_cn/products/system/storage/2u/ssg-229j-5bu24jbf.
[103] Kuniyasu Suzaki, Kenta Nakajima, Tsukasa Oi, and Akira Tsukamoto. 2020. Library implementation and performance analysis of GlobalPlatform TEE Internal API for Intel SGX and RISC-V Keystone. In *2020 IEEE 19th International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom)*. IEEE, 1200–1208.
[104] Synopsys. 2024. Protecting Data over PCIe CXL in Cloud Computing. https://www.synopsys.com/designware-ip/technical-bulletin/security-ide-ip.html.
[105] Synopsys. 2024. Synopsys IDE Security IP Modules for PCI Express 5. https://www.synopsys.com/dw/doc.php/ds/s/ide-pcie-integrity-data-encryption-security-module.pdf.
[106] synopsys. 2024. TEE Device Interface Secure Protocol (TDISP) for PCIe and CXL. https://www.synopsys.com/designware-ip/technical-bulletin/tee-device-interface-secure-protocol.html.
[107] Yupeng Tang, Ping Zhou, Wenhui Zhang, Henry Hu, Qirui Yang, Hao Xiang, Tongping Liu, Jiaxin Shan, Ruoyun Huang, Cheng Zhao, et al. 2024. Exploring Performance and Cost Optimization with ASIC-Based CXL Memory. In *Proceedings of the Nineteenth European Conference on Computer Systems*. 818–833.
[108] Maxio Technology. 2024. About Lianyun Technology (Hangzhou) Co. Initial Public Offering and Listing on Technology and Innovation Board (TECHNOLOGY) Response to the Audit Inquiry Letter on the Filing Documents. https://static.sse.com.cn/stock/disclosure/announcement/c/202405/001363_20240523_AP3M.pdf.
[109] Techpowerup. 2022. DapuStor Haishen3 H3000 1 TB. https://www.techpowerup.com/ssd-specs/dapustor-haishen3-h3000-1-tb.d1647.
[110] Techpowerup. 2023. DapuStor Haishen5 H5100 7.5 TB. https://www.techpowerup.com/ssd-specs/dapustor-haishen5-h5100-7-5-tb.d1785.
[111] TechPowerUp. 2024. FlumeIO F5900 3.8 TB. https://www.techpowerup.com/ssd-specs/flumeio-f5900-3-8-tb.d2138.
[112] Techpowerup. 2024. Memblaze P7946 6 TB. https://www.techpowerup.com/ssd-specs/memblaze-p7946-6-tb.d1582.
[113] TechPowerUp. 2024. Phison Pascari X200E 1.6 TB. https://www.techpowerup.com/ssd-specs/phison-pascari-x200e-1-6-tb.d2017.
[114] TechPowerUp. 2024. Samsung 980. https://www.techpowerup.com/ssd-specs/samsung-980-1-tb.d1916.
[115] TechPowerUp. 2024. Solidigm D7-PS1010 1.9 TB. https://www.techpowerup.com/ssd-specs/phison-pascari-x200e-1-6-tb.d20178.
[116] Huangshi Tian, Yunchuan Zheng, and Wei Wang. 2019. Characterizing and synthesizing task dependencies of data-parallel jobs in alibaba cloud. In *Proceedings of the ACM Symposium on Cloud Computing*. 139–151.
[117] TRENDFORCE. 2024. NAND Flash Price Trends. https://www.trendforce.com/price/flash.
[118] Juan Wang, Zhi Hong, Yuhan Zhang, and Yier Jin. 2017. Enabling security-enhanced attestation with Intel SGX for remote terminal and IoT. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 37, 1 (2017), 88–96.
[119] Jiapin Wang, Xiangping Zhang, Chenlei Tang, Xiang Chen, and Tao Lu. 2024. LMB: Augmenting PCIe Devices with CXL-Linked Memory Buffer. *arXiv preprint arXiv:2406.02039* (2024).
[120] Yuyue Wang, Xiurui Pan, Yuda An, Jie Zhang, and Glenn Reinman. 2024. BeaconGNN: Large-Scale GNN Acceleration with Out-of-Order Streaming In-Storage Computing. In *2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 330–344.
[121] Ofir Weisse, Valeria Bertacco, and Todd Austin. 2017. Regaining lost cycles with HotCalls: A fast interface for SGX secure enclaves. *ACM SIGARCH Computer Architecture News* 45, 2, 81–93.
[122] Chun-Feng Wu, Yuan-Hao Chang, Ming-Chang Yang, and Tei-Wei Kuo. 2024. How to steal CPU idle time when synchronous I/O mode becomes promising. In *Proceedings of the 61st ACM/IEEE Design Automation Conference*. 1–6.
[123] Jisoo Yang, Dave B Minturn, and Frank T Hady. 2012. When poll is better than interrupt.. In *FAST*, Vol. 12. 3–3.
[124] Pan Yang, Ni Xue, Yuqi Zhang, Yangxu Zhou, Li Sun, Wenwen Chen, Zhonggang Chen, Wei Xia, Junke Li, and Kihyoun Kwon. 2019. Reducing garbage collection overhead in {SSD} based on workload prediction. In *11th USENIX Workshop on Hot Topics in Storage and File Systems (HotStorage 19)*.
[125] Sean Yang. 2017. Improving the design of DRAM-Less PCIe SSD. *Flash Memory Summit* (2017).
[126] Shao-Peng Yang, Minjae Kim, Sanghyun Nam, Juhyung Park, Jin-Yong Choi, Eyee Hyun Nam, Eunji Lee, Sungjin Lee, and Bryan S Kim. 2023. Overcoming the Memory Wall with {CXL-Enabled}{SSDs}. In *2023 USENIX Annual Technical Conference (USENIX ATC 23)*. 601–617.
[127] Shushu Yi, Shaocong Sun, Li Peng, Yingbo Sun, Ming-Chang Yang, Zhichao Cao, Qiao Li, Myoungsoo Jung, Ke Zhou, and Jie Zhang. 2024. BIZA: Design of Self-Governing Block-Interface ZNS AFA for Endurance and Performance. In *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles*. 313–329.
[128] Jie Zhang and Myoungsoo Jung. 2020. Zng: Architecting gpu multi-processors with new flash for scalable data analysis. In *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 1064–1075.
[129] Jie Zhang, Miryeong Kwon, Michael Swift, and Myoungsoo Jung. 2020. Scalable parallel flash firmware for many-core architectures. In *18th USENIX Conference on File and Storage Technologies (FAST 20)*. 121–136.
[130] Jian Zhang, Yujie Ren, Marie Nguyen, Changwoo Min, and Sudarsun Kannan. 2024. {OmniCache}: Collaborative Caching for Near-storage Accelerators. In *22nd USENIX Conference on File and Storage Technologies (FAST 24)*. 35–50.
[131] Xiaoyi Zhang, Feng Zhu, Shu Li, Kun Wang, Wei Xu, and Dengcai Xu. 2021. Optimizing performance for open-channel ssds in cloud storage system. In *2021 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. IEEE, 902–911.
[132] Yuhong Zhong, Haoyu Li, Yu Jian Wu, Ioannis Zarkadas, Jeffrey Tao, Evan Mesterhazy, Michael Makris, Junfeng Yang, Amy Tai, Ryan Stutsman, et al. 2022. {XRP}:{In-Kernel} Storage Functions with {EBPF}. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. 375–393.
[133] Qiang Zou, Yifeng Zhu, Jianxi Chen, Yuhui Deng, and Xiao Qin. 2023. Characterization of I/O Behaviors in Cloud Storage Workloads. *IEEE Trans. Comput.* 72, 10 (2023), 2726–2739.